

```
/* ===== 源文件: prescription-core.js ===== */
(function (global) {
  'use strict';

  function formatPrice(val) {
    if (val === '' || val === null || val === undefined) return '';
    const num = parseFloat(val);
    if (isNaN(num)) return '';
    return num.toFixed(2);
  }

  function escapeHtml(str) {
    if (!str) return '';
    return String(str)
      .replace(/&/g, '&amp;')
      .replace(/</g, '&lt;')
      .replace(/>/g, '&gt;')
      .replace(/"/g, '&quot;')
      .replace(/'/g, '&#39;');
  }

  function sortPrescriptionsByTimeDesc(list) {
    if (!Array.isArray(list)) return list;
    return list.sort((a, b) => {
      const timeA = new Date(a.createdAt || a.date || a.id || 0).getTime();
      const timeB = new Date(b.createdAt || b.date || b.id || 0).getTime();
      return timeB - timeA;
    });
  }

  function sortPrescriptionsByNoDesc(list) {
    if (!Array.isArray(list)) return list;
    return list.sort((a, b) => {
      const noA = a.outpatientNo || a.prescriptionNo || '';
      const noB = b.outpatientNo || b.prescriptionNo || '';
      return noB.localeCompare(noA);
    });
  }

  function createEmptyMedicine() {
    return { code: '', name: '', dosage: '', unit: 'g', price: '', total: '' };
  }

  function calculateItemTotal(price, dosage) {
    const p = parseFloat(price) || 0;
    const d = parseFloat(dosage) || 0;
    return p * d;
  }

  function calculateMedicineTotal(items) {
    if (!Array.isArray(items)) return 0;
    return items.reduce((sum, item) => {
      if (!item || !item.name) return sum;
      return sum + (parseFloat(item.total) || calculateItemTotal(item.price, item.dosage));
    }, 0);
  }

  function calculatePrescriptionTotal(items, doseCount, registrationFee) {
    const medicineTotal = calculateMedicineTotal(items);
    const dose = parseInt(doseCount) || 1;
    const fee = parseFloat(registrationFee) || 0;
    return medicineTotal * dose + fee;
  }

  function calculatePerDosePrice(items) {
    return calculateMedicineTotal(items);
  }
})
```

```
function validatePrescription(record) {
  if (!record) return { valid: false, error: '处方数据为空' };
  if (!record.patientName) return { valid: false, error: '请输入患者姓名' };
  if (!record.items || !Array.isArray(record.items) || record.items.length === 0) {
    return { valid: false, error: '请至少添加一味药品' };
  }
  const hasValidItem = record.items.some(item => item && item.name);
  if (!hasValidItem) return { valid: false, error: '请至少添加一味药品' };
  if (!record.doseCount || parseInt(record.doseCount) <= 0) {
    return { valid: false, error: '剂数必须大于0' };
  }
  return { valid: true };
}

function buildPrescriptionRecord(formData) {
  const now = new Date().toISOString();
  const items = (formData.items || []).filter(item => item && item.name).map(item => ({
    code: item.code || '',
    name: item.name || '',
    dosage: item.dosage || '',
    unit: item.unit || 'g',
    price: parseFloat(item.price) || 0,
    total: parseFloat(item.total) || calculateItemTotal(item.price, item.dosage)
  }));
  const doseCount = parseInt(formData.doseCount) || 7;
  const registrationFee = parseFloat(formData.registrationFee) || 0;
  const medicineTotal = calculateMedicineTotal(items);
  const totalAmount = medicineTotal * doseCount + registrationFee;
  return {
    id: formData.id || Date.now(),
    date: formData.date || new Date().toISOString().split('T')[0],
    patientName: formData.patientName || '',
    patientGender: formData.patientGender || '',
    patientAge: formData.patientAge || '',
    patientPhone: formData.patientPhone || '',
    patientAddress: formData.patientAddress || '',
    doctorName: formData.doctorName || '',
    medicalHistory: formData.medicalHistory || '',
    diagnosis: formData.diagnosis || '',
    items: items,
    doseCount: doseCount,
    medicineTotal: medicineTotal,
    perDosePrice: medicineTotal,
    registrationFee: registrationFee,
    totalAmount: totalAmount,
    createdBy: formData.createdBy || 'unknown',
    createdAt: formData.createdAt || now,
    updatedAt: now
  };
}

function loadMedicineFrequency(storageKey) {
  const key = storageKey || 'local_medicineFrequency';
  try {
    const saved = global.localStorage ? global.localStorage.getItem(key) : null;
    return saved ? JSON.parse(saved) : {};
  } catch (e) {
    return {};
  }
}
```

```
function saveMedicineFrequency(frequency, storageKey) {
  const key = storageKey || 'local_medicineFrequency';
  try {
    if (global.localStorage) {
      global.localStorage.setItem(key, JSON.stringify(frequency));
    }
  } catch (e) {
    console.warn('[PrescriptionCore] 保存药品频率失败:', e);
  }
}

function updateMedicineFrequency(items, frequency, storageKey) {
  const freq = frequency || {};
  (items || []).forEach(item => {
    if (item && item.name) {
      freq[item.name] = (freq[item.name] || 0) + 1;
    }
  });
  if (storageKey !== false) saveMedicineFrequency(freq, storageKey);
  return freq;
}

function generateOfflinePrescriptionNo(date, seq) {
  const d = date || new Date();
  const yy = String(d.getFullYear()).slice(2);
  const mm = String(d.getMonth() + 1).padStart(2, '0');
  const dd = String(d.getDate()).padStart(2, '0');
  const seqStr = String(seq || 1).padStart(2, '0');
  return yy + mm + dd + seqStr;
}

function generateStatisticsNo(date, total) {
  const d = date || new Date();
  const yy = String(d.getFullYear()).slice(2);
  const count = parseInt(total) || 0;
  return yy + String(count + 1).padStart(6, '0');
}

function generatePrescriptionSummary(record) {
  if (!record) return '';
  const date = record.date || '';
  const name = record.patientName || '';
  const diagnosis = record.diagnosis || '';
  const fee = formatPrice(record.totalAmount);
  return date + ' - ' + name + ' | ' + diagnosis + (fee ? ' | 诊费:' + fee + '元' : '');
}

const PrescriptionCore = {
  formatPrice,
  escapeHtml,
  sortPrescriptionsByTimeDesc,
  sortPrescriptionsByNoDesc,
  createEmptyMedicine,
  calculateItemTotal,
  calculateMedicineTotal,
  calculatePrescriptionTotal,
  calculatePerDosePrice,
  validatePrescription,
  buildPrescriptionRecord,
  loadMedicineFrequency,
  saveMedicineFrequency,
  updateMedicineFrequency,
  generateOfflinePrescriptionNo,
```

```
generateStatisticsNo,
generatePrescriptionSummary
});
global.PrescriptionCore = PrescriptionCore;
})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
/* ===== 源文件: medicine-dict.js ===== */
(function (global) {
  'use strict';
  const MedicineDict = {
    _medicines: null,
    _pinyinIndex: null,
    init(medicines) {
      this._medicines = medicines || [];
      this._buildIndex();
    },
    _buildIndex() {
      if (!this._medicines) return;
      this._pinyinIndex = this._medicines.map(m => ({
        name: (m.name || '').toLowerCase(),
        code: (m.code || '').toLowerCase(),
        pinyin: (m.pinyin || '').toLowerCase(),
        original: m
      }));
    },
    search(keyword, limit) {
      if (!this._pinyinIndex || !keyword) return [];
      limit = limit || 20;
      const kw = keyword.toLowerCase().trim();
      const exact = [];
      const prefix = [];
      const contains = [];
      for (const item of this._pinyinIndex) {
        if (item.name === kw || item.code === kw) {
          exact.push(item.original);
        } else if (item.name.startsWith(kw) || item.code.startsWith(kw)) {
          prefix.push(item.original);
        } else if (item.name.includes(kw) || item.code.includes(kw) || item.pinyin.includes(kw)) {
          contains.push(item.original);
        }
        if (exact.length + prefix.length + contains.length >= limit * 2) break;
      }
      return [...exact, ...prefix, ...contains].slice(0, limit);
    },
    findByCode(code) {
      if (!this._pinyinIndex || !code) return null;
      const lc = code.toLowerCase().trim();
      const match = this._pinyinIndex.find(m => m.code === lc);
      return match ? match.original : null;
    },
    findByName(name) {
      if (!this._pinyinIndex || !name) return null;
      const lc = name.toLowerCase().trim();
      const match = this._pinyinIndex.find(m => m.name === lc);
      return match ? match.original : null;
    },
    formatMedicine(m) {
      if (!m) return '';
      const parts = [m.name];
    }
  };
});
```

```
    if (m.unit) parts.push(`${m.unit}`);
    if (m.price) parts.push(`¥${m.price}`);
    return parts.join(' ');
  },
  calculateTotal(medicines) {
    if (!medicines || !medicines.length) return 0;
    return medicines.reduce((sum, m) => {
      const qty = parseFloat(m.quantity) || 0;
      const price = parseFloat(m.price) || 0;
      return sum + (qty * price);
    }, 0);
  },
  generateId() {
    return 'med_' + Date.now() + '_' + Math.floor(Math.random() * 1000);
  },
  validate(medicine) {
    const errors = [];
    if (!medicine.name || !medicine.name.trim()) errors.push('药名不能为空');
    if (!medicine.unit) errors.push('单位不能为空');
    if (medicine.price !== undefined && isNaN(parseFloat(medicine.price))) {
      errors.push('单价必须是数字');
    }
    return errors;
  },
  loadFromStorage(key) {
    key = key || 'medicine_library';
    try {
      const data = localStorage.getItem(key);
      const medicines = data ? JSON.parse(data) : [];
      this.init(medicines);
      return medicines;
    } catch (e) {
      console.warn('[DBG] 加载药材库失败:', e);
      this.init([]);
      return [];
    }
  },
  saveToStorage(medicines, key) {
    key = key || 'medicine_library';
    try {
      localStorage.setItem(key, JSON.stringify(medicines));
      this.init(medicines);
      return true;
    } catch (e) {
      console.error('[DBG] 保存药材库失败:', e);
      return false;
    }
  },
  getAll() {
    return this._medicines || [];
  },
  filterByCategory(category) {
    if (!this._medicines) return [];
    if (!category) return this._medicines;
    return this._medicines.filter(m => m.category === category);
  }
};

global.MedicineDict = MedicineDict;
```

```

})(typeof window !== 'undefined' ? window : this);
/* ===== 源文件: auth-core.js ===== */
(function (global) {
  'use strict';
  const PASSWORD_SALT = 'bnzc_prescription_salt_v1';
  const SESSION_TIMEOUT_MS = 8 * 60 * 60 * 1000; // 8小时
  const CLOUD_API_BASE = 'https://tcm-prescription-system.pages.dev/api';
  function _deriveRuntimeKey() {
    const env = (typeof location !== 'undefined' && location.hostname) ? location.hostname : 'local';
    const lang = (typeof navigator !== 'undefined' && navigator.language) ? navigator.language : '';
    let hash = 0;
    const src = PASSWORD_SALT + '|' + env + '|' + lang;
    for (let i = 0; i < src.length; i++) {
      hash = ((hash << 5) - hash) + src.charCodeAt(i);
      hash = hash & hash;
    }
    return PASSWORD_SALT + '_' + Math.abs(hash).toString(36);
  }
  const RUNTIME_KEY = _deriveRuntimeKey();
  let _masterKey = null;
  function setMasterKey(key) {
    _masterKey = key ? String(key) : null;
    if (_masterKey) {
      try { console.log('[AuthCore] masterKey 已注入，密码哈希将使用派生盐'); } catch (e) { }
    }
  }
  function getMasterKey() {
    return _masterKey;
  }
  function getEffectiveSalt() {
    return _masterKey ? (PASSWORD_SALT + ':' + _masterKey) : PASSWORD_SALT;
  }
  const SafeStorageBridge = {
    _availableCache: null,
    async available() {
      if (this._availableCache !== null) return this._availableCache;
      try {
        if (global.electronAPI && typeof global.electronAPI.safeStorageAvailable === 'function') {
          this._availableCache = await global.electronAPI.safeStorageAvailable();
        } else {
          this._availableCache = false;
        }
      } catch (e) {
        console.warn('safeStorage 检测异常:', e);
        this._availableCache = false;
      }
      return this._availableCache;
    },
    async encrypt(plaintext) {
      if (!plaintext) return null;
      try {
        if (!(await this.available())) return null;
        if (!global.electronAPI || typeof global.electronAPI.encryptString !== 'function') return null;
        return await global.electronAPI.encryptString(String(plaintext));
      } catch (e) {
        console.warn('safeStorage 加密异常:', e);
        return null;
      }
    }
  }

```

```
    },
    async decrypt(encryptedBase64) {
      if (!encryptedBase64) return null;
      try {
        if (!(await this.available())) return null;
        if (!global.electronAPI || typeof global.electronAPI.decryptString !== 'function') return null;
        return await global.electronAPI.decryptString(String(encryptedBase64));
      } catch (e) {
        console.warn('safeStorage 解密异常:', e);
        return null;
      }
    }
  };
const KEY_MIGRATION = {
  'cloud_currentUser': 'auth:currentUser',
  'cloud_isLoggedIn': 'auth:isLoggedIn',
  'currentUser': 'auth:currentUser',
  'isLoggedIn': 'auth:isLoggedIn',
  'user_login_data': 'auth:loginData',
  'cloud_rememberedUsers': 'auth:rememberedUsers',
  'cloud_rememberedUsername': 'auth:rememberedUsername',
  'local_rememberedUsername': 'auth:rememberedUsername',
  'local_rememberedUsers': 'auth:rememberedUsers',
  'rememberedUsername': 'auth:rememberedUsername',
  'rememberedUsers': 'auth:rememberedUsers'
};
const StorageAdapter = {
  _isCapacitor() {
    return typeof global.Capacitor !== 'undefined' &&
      global.Capacitor.Plugins &&
      global.Capacitor.Plugins.Preferences;
  },
  async getItem(key) {
    if (this._isCapacitor()) {
      const { value } = await global.Capacitor.Plugins.Preferences.get({ key });
      return value;
    }
    return global.localStorage ? global.localStorage.getItem(key) : null;
  },
  async setItem(key, value) {
    if (this._isCapacitor()) {
      await global.Capacitor.Plugins.Preferences.set({ key, value });
      return;
    }
    if (global.localStorage) global.localStorage.setItem(key, value);
  },
  async removeItem(key) {
    if (this._isCapacitor()) {
      await global.Capacitor.Plugins.Preferences.remove({ key });
      return;
    }
    if (global.localStorage) global.localStorage.removeItem(key);
  },
  setSessionItem(key, value) {
    if (typeof global.sessionStorage !== 'undefined') {
      global.sessionStorage.setItem(key, value);
    }
  },
};
```

```

getSessionItem(key) {
  if (typeof global.sessionStorage !== 'undefined') {
    return global.sessionStorage.getItem(key);
  }
  return null;
},
removeSessionItem(key) {
  if (typeof global.sessionStorage !== 'undefined') {
    global.sessionStorage.removeItem(key);
  }
}
};
let _migrated = false;
async function migrateOldKeys() {
  if (_migrated) return;
  _migrated = true;
  for (const [oldKey, newKey] of Object.entries(KEY_MIGRATION)) {
    try {
      const value = await StorageAdapter.getItem(oldKey);
      if (value !== null && value !== undefined) {
        const newValue = await StorageAdapter.getItem(newKey);
        if (newValue === null || newValue === undefined) {
          await StorageAdapter.setItem(newKey, value);
        }
      }
    } catch (e) { }
  }
  for (const [oldKey, newKey] of Object.entries(KEY_MIGRATION)) {
    try {
      const value = StorageAdapter.getSessionItem(oldKey);
      if (value !== null) {
        const newValue = StorageAdapter.getSessionItem(newKey);
        if (newValue === null) {
          StorageAdapter.setSessionItem(newKey, value);
        }
      }
    } catch (e) { }
  }
}
function sha256PureJS(message) {
  const K = [0x428a2f98, 0x71374491, 0xb5c0fbcf, 0xe9b5dba5, 0x3956c25b, 0x59f111f1, 0x923f82a4, 0xab1c5ed5,
    0xd807aa98, 0x12835b01, 0x243185be, 0x550c7dc3, 0x72be5d74, 0x80deb1fe, 0x9bdc06a7, 0xc19bf174,
    0xe49b69c1, 0xefbe4786, 0x0fc19dc6, 0x240ca1cc, 0x2de92c6f, 0x4a7484aa, 0x5cb0a9dc, 0x76f988da,
    0x983e5152, 0xa831c66d, 0xb00327c8, 0xbf597fc7, 0xc6e00bf3, 0xd5a79147, 0x06ca6351, 0x14292967,
    0x27b70a85, 0x2e1b2138, 0x4d2c6dfc, 0x53380d13, 0x650a7354, 0x766a0abb, 0x81c2c92e, 0x92722c85,
    0xa2bfe8a1, 0xa81a664b, 0xc24b8b70, 0xc76c51a3, 0xd192e819, 0xd6990624, 0xf40e3585, 0x106aa070,
    0x19a4c116, 0x1e376c08, 0x2748774c, 0x34b0bcb5, 0x391c0cb3, 0x4ed8aa4a, 0x5b9cca4f, 0x682e6fff,
    0x748f82ee, 0x78a5636f, 0x84c87814, 0x8cc70208, 0x90befffa, 0xa4506ceb, 0xbef9a3f7, 0xc67178f2];
  function rotateRight(n, s) { return (n >>> s) | (n << (32 - s)); }
  function sigma0(n) { return rotateRight(n, 7) ^ rotateRight(n, 18) ^ (n >>> 3); }
  function sigma1(n) { return rotateRight(n, 17) ^ rotateRight(n, 19) ^ (n >>> 10); }
  function Sigma0(n) { return rotateRight(n, 2) ^ rotateRight(n, 13) ^ rotateRight(n, 22); }
  function Sigma1(n) { return rotateRight(n, 6) ^ rotateRight(n, 11) ^ rotateRight(n, 25); }
  function ch(x, y, z) { return (x & y) ^ (~x & z); }
  function maj(x, y, z) { return (x & y) ^ (x & z) ^ (y & z); }
  const bytes = new Uint8Array(message.length);
  for (let i = 0; i < message.length; i++) bytes[i] = message.charCodeAt(i);
  const originalLen = bytes.length;

```

```

const paddingLen = 64 - ((originalLen + 8) % 64);
const padded = new Uint8Array(originalLen + paddingLen + 8);
padded.set(bytes);
padded[originalLen] = 0x80;
let len = originalLen * 8;
for (let i = 7; i >= 0; i--) {
  padded[padded.length - 1 - i] = len & 0xff;
  len >>= 8;
}
const w = new Uint32Array(64);
let h0 = 0x6a09e667, h1 = 0xbb67ae85, h2 = 0x3c6ef372, h3 = 0xa54ff53a;
let h4 = 0x510e527f, h5 = 0x9b05688c, h6 = 0x1f83d9ab, h7 = 0x5be0cd19;
for (let block = 0; block < padded.length; block += 64) {
  for (let i = 0; i < 16; i++) {
    w[i] = (padded[block + i * 4] << 24) | (padded[block + i * 4 + 1] << 16) |
      (padded[block + i * 4 + 2] << 8) | padded[block + i * 4 + 3];
  }
  for (let i = 16; i < 64; i++) {
    w[i] = (sigma1(w[i - 2]) + w[i - 7] + sigma0(w[i - 15]) + w[i - 16]) >>> 0;
  }
  let a = h0, b = h1, c = h2, d = h3, e = h4, f = h5, g = h6, h = h7;
  for (let i = 0; i < 64; i++) {
    const T1 = (h + Sigma1(e) + ch(e, f, g) + K[i] + w[i]) >>> 0;
    const T2 = (Sigma0(a) + maj(a, b, c)) >>> 0;
    h = g; g = f; f = e; e = (d + T1) >>> 0;
    d = c; c = b; b = a; a = (T1 + T2) >>> 0;
  }
  h0 = (h0 + a) >>> 0; h1 = (h1 + b) >>> 0; h2 = (h2 + c) >>> 0; h3 = (h3 + d) >>> 0;
  h4 = (h4 + e) >>> 0; h5 = (h5 + f) >>> 0; h6 = (h6 + g) >>> 0; h7 = (h7 + h) >>> 0;
}
const hash = [h0, h1, h2, h3, h4, h5, h6, h7];
return hash.map(n => n.toString(16).padStart(8, '0')).join('');
}
async function hashPassword(password) {
  if (!password) return '';
  try {
    const data = new TextEncoder().encode(PASSWORD_SALT + password);
    const hashBuffer = await crypto.subtle.digest('SHA-256', data);
    return Array.from(new Uint8Array(hashBuffer))
      .map(b => b.toString(16).padStart(2, '0')).join('');
  } catch (e) {
    return sha256PureJS(PASSWORD_SALT + password);
  }
}
async function hashPasswordWithUser(password, username) {
  if (!password) return '';
  const userSalt = username ? (PASSWORD_SALT + ':' + username) : PASSWORD_SALT;
  try {
    const data = new TextEncoder().encode(userSalt + password);
    const hashBuffer = await crypto.subtle.digest('SHA-256', data);
    return Array.from(new Uint8Array(hashBuffer))
      .map(b => b.toString(16).padStart(2, '0')).join('');
  } catch (e) {
    return sha256PureJS(userSalt + password);
  }
}
function isPasswordHashed(pwd) {
  return typeof pwd === 'string' && pwd.length === 64 && /^[a-f0-9]{64}$/.test(pwd);
}

```

```

}
async function verifyPassword(inputPassword, storedPassword, username) {
  if (!storedPassword) return false;
  if (isPasswordHashed(storedPassword)) {
    if (username) {
      const enhancedHash = await hashPasswordWithUser(inputPassword, username);
      if (storedPassword === enhancedHash) return true;
    }
    return storedPassword === await hashPassword(inputPassword);
  }
  return storedPassword === inputPassword;
}

function findUserByIdentifier(users, identifier) {
  if (!Array.isArray(users) || !identifier) return null;
  const trimmed = String(identifier).trim();
  if (!trimmed) return null;
  let user = users.find(u => u && u.username === trimmed);
  if (!user) {
    user = users.find(u => u && u.phone && String(u.phone) === trimmed);
  }
  return user || null;
}

function validateUsername(username) {
  if (!username || typeof username !== 'string') {
    return { valid: false, error: '用户名不能为空' };
  }
  let trimmed = username.trim();
  if (trimmed.length < 2 || trimmed.length > 30) {
    return { valid: false, error: '用户名长度需 2-30 个字符' };
  }
  trimmed = trimmed.replace(/[\uff01-\uff5e]/g, function(ch) {
    return String.fromCharCode(ch.charCodeAt(0) - 0xfee0);
  });
  if (/[\x00-\x1f\x7f\u200b-\u200f\u202a-\u202e\ufeff]/.test(trimmed)) {
    return { valid: false, error: '用户名不能包含控制字符或不可见字符' };
  }
  if (/[;'"\\<>|&`$#@!%^(*)+=\[\]\{\}~]/.test(trimmed)) {
    return { valid: false, error: '用户名不能包含特殊符号（标点/引号/括号等）' };
  }
  return { valid: true, username: trimmed };
}

function validateAdminUsername(username) {
  const base = validateUsername(username);
  if (!base.valid) return base;
  const usernameForAdmin = base.username;
  if (/[\u4e00-\u9fa5]/.test(usernameForAdmin)) {
    return base;
  }
  if (usernameForAdmin.startsWith('admin_')) {
    if (!/^admin_[a-z][a-z0-9]{1,11}$/i.test(usernameForAdmin)) {
      return {
        valid: false,
        error: '管理员简码账号必须为 admin_诊所简码 格式（如 admin_hkt），简码为 2-12 位小写字母和数字'
      };
    }
  }
  return base;
}

```

```

async function encryptPassword(password) {
  if (!password) return null;
  const safeEnc = await SafeStorageBridge.encrypt(password);
  if (safeEnc) return 'SAFE:' + safeEnc;
  const key = RUNTIME_KEY;
  let result = '';
  for (let i = 0; i < password.length; i++) {
    result += String.fromCharCode(password.charCodeAt(i) ^ key.charCodeAt(i % key.length));
  }
  return 'PWDv2:' + btoa(unescape(encodeURIComponent(result)));
}

async function decryptPassword(stored) {
  if (!stored || typeof stored !== 'string') return null;
  if (stored.startsWith('SAFE:')) {
    try {
      const decrypted = await SafeStorageBridge.decrypt(stored.substring(5));
      if (decrypted !== null) return decrypted;
      console.warn('safeStorage 解密失败: 可能为跨用户/跨机器迁移, 请重新输入密码');
      return null;
    } catch (e) {
      console.error('safeStorage 密码解密失败:', e);
      return null;
    }
  }
  if (stored.startsWith('PWDv2:')) {
    try {
      const text = decodeURIComponent(escape(atob(stored.substring(6))));
      const key = RUNTIME_KEY;
      let result = '';
      for (let i = 0; i < text.length; i++) {
        result += String.fromCharCode(text.charCodeAt(i) ^ key.charCodeAt(i % key.length));
      }
      return result;
    } catch (e) {
      console.error('密码解密失败:', e);
      return null;
    }
  }
  if (stored.startsWith('PWDv1:')) {
    try {
      const text = decodeURIComponent(escape(atob(stored.substring(6))));
      const key = PASSWORD_SALT;
      let result = '';
      for (let i = 0; i < text.length; i++) {
        result += String.fromCharCode(text.charCodeAt(i) ^ key.charCodeAt(i % key.length));
      }
      return result;
    } catch (e) {
      console.error('密码解密失败:', e);
      return null;
    }
  }
  return stored;
}

async function saveRememberedPassword(password) {
  try {
  } catch (e) { }
}

```

```
async function getRememberedPassword() {
    return null;
}
async function clearRememberedPassword() {
    try {
        await StorageAdapter.removeItem('auth:savedPassword');
    } catch (e) { }
}
async function encryptUsers(users) {
    const json = JSON.stringify(users);
    const safeEnc = await SafeStorageBridge.encrypt(json);
    if (safeEnc) return 'SAFE:' + safeEnc;
    const key = RUNTIME_KEY;
    let result = '';
    for (let i = 0; i < json.length; i++) {
        result += String.fromCharCode(json.charCodeAt(i) ^ key.charCodeAt(i % key.length));
    }
    return 'XORv2:' + btoa(unescape(encodeURIComponent(result)));
}
async function decryptUsers(stored) {
    if (!stored || typeof stored !== 'string') return stored;
    if (stored.startsWith('SAFE:')) {
        try {
            const decrypted = await SafeStorageBridge.decrypt(stored.substring(5));
            if (decrypted !== null) {
                try { return JSON.parse(decrypted); }
                catch (e) { console.error('解密后 JSON 解析失败:', e); return null; }
            }
            console.warn('safeStorage 用户列表解密失败：可能为跨用户/跨机器迁移');
            return null;
        } catch (e) {
            console.error('safeStorage 用户列表解密失败:', e);
            return null;
        }
    }
}
if (stored.startsWith('XORv2:')) {
    try {
        const text = decodeURIComponent(escape(atob(stored.substring(6))));
        const key = RUNTIME_KEY;
        let result = '';
        for (let i = 0; i < text.length; i++) {
            result += String.fromCharCode(text.charCodeAt(i) ^ key.charCodeAt(i % key.length));
        }
        return JSON.parse(result);
    } catch (e) {
        console.error('解密用户列表失败:', e);
        return null;
    }
}
if (stored.startsWith('XORv1:')) {
    try {
        const text = decodeURIComponent(escape(atob(stored.substring(6))));
        const key = PASSWORD_SALT;
        let result = '';
        for (let i = 0; i < text.length; i++) {
            result += String.fromCharCode(text.charCodeAt(i) ^ key.charCodeAt(i % key.length));
        }
        return JSON.parse(result);
    }
}
```

```

    } catch (e) {
      console.error('解密用户列表失败:', e);
      return null;
    }
  }
  return stored;
}

function resolveAllowedMode(user) {
  if (!user) return 'local';
  if (user.role === 'platform_admin') return 'platform_admin';
  return user.allowedMode || 'both';
}

function isAdmin(user) {
  if (!user) return false;
  return user.role === 'admin' || user.role === 'clinic_admin' || user.role === 'platform_admin';
}

function isClinicAdmin(user) {
  if (!user) return false;
  return user.role === 'admin' || user.role === 'clinic_admin';
}

function isPlatformAdmin(user) {
  if (!user) return false;
  return user.role === 'platform_admin';
}

function buildAuthPayload(user) {
  if (!user) return null;
  return btoa(unescape(encodeURIComponent(JSON.stringify({
    username: user.username,
    role: user.role || 'doctor',
    clinicId: user.clinicId || null
  })))));
}

let _sessionMonitorTimer = null;
let _sessionLogoutCallback = null;
function startSessionMonitor(logoutCallback) {
  if (_sessionMonitorTimer) {
    clearInterval(_sessionMonitorTimer);
    _sessionMonitorTimer = null;
  }
  _sessionLogoutCallback = logoutCallback || null;
  const CHECK_INTERVAL_MS = 5 * 60 * 1000; // 每5分钟检查一次（省电+及时）
  _sessionMonitorTimer = setInterval(async function() {
    try {
      const session = await checkSession();
      if (!session.valid && session.reason === 'session_timeout') {
        if (typeof _sessionLogoutCallback === 'function') {
          try { await _sessionLogoutCallback('session_timeout'); } catch (e) {}
        }
        try { await logout(); } catch (e) {}
        try { console.log('[AuthCore] 会话超时，已自动登出（8小时未刷新）'); } catch (e) {}
      }
    } catch (e) {
      try { console.warn('[AuthCore] 会话监控检查异常:', e); } catch (e2) {}
    }
  }, CHECK_INTERVAL_MS);
  try { console.log('[AuthCore] 会话监控已启动（每5分钟检查一次，8小时超时自动登出）'); } catch (e) {}
}

function stopSessionMonitor() {

```

```

    if (_sessionMonitorTimer) {
      clearInterval(_sessionMonitorTimer);
      _sessionMonitorTimer = null;
      try { console.log('[AuthCore] 会话监控已停止'); } catch (e) {}
    }
    _sessionLogoutCallback = null;
  }
  async function checkSession() {
    try {
      const loginData = await StorageAdapter.getItem('auth:loginData');
      if (loginData) {
        const { loginTime } = JSON.parse(loginData);
        if (loginTime && Date.now() - loginTime > SESSION_TIMEOUT_MS) {
          await logout();
          return { valid: false, reason: 'session_timeout' };
        }
      }
    } catch (e) { }
    try {
      const oldLoginData = await StorageAdapter.getItem('user_login_data');
      if (oldLoginData) {
        const { loginTime } = JSON.parse(oldLoginData);
        if (loginTime && Date.now() - loginTime > SESSION_TIMEOUT_MS) {
          await logout();
          return { valid: false, reason: 'session_timeout' };
        }
      }
    } catch (e) { }
    const isLoggedInIn = await StorageAdapter.getItem('auth:isLoggedInIn');
    const userStr = await StorageAdapter.getItem('auth:currentUser');
    if (isLoggedInIn === 'true' && userStr) {
      try {
        return { valid: true, user: JSON.parse(userStr) };
      } catch (e) { }
    }
    const oldLoggedIn = await StorageAdapter.getItem('cloud_isLoggedIn') ||
      await StorageAdapter.getItem('isLoggedInIn');
    const oldUserStr = await StorageAdapter.getItem('cloud_currentUser') ||
      await StorageAdapter.getItem('currentUser');
    if (oldLoggedIn === 'true' && oldUserStr) {
      try {
        return { valid: true, user: JSON.parse(oldUserStr) };
      } catch (e) { }
    }
    return { valid: false, reason: 'not_logged_in' };
  }
  const cloudAdapter = {
    async authenticate(username, password) {
      try {
        const fetchFn = global.cloudFetch || global.fetch;
        const response = await fetchFn(`${CLOUD_API_BASE}/users?login=true`, {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({ username, password })
        });
        const data = (response && typeof response.json() === 'function')
          ? await response.json()
          : response;
      }
    }
  }

```

```

    if (!data || !data.success || !data.user) {
      return { success: false, error: (data && data.error) || '手机号/用户名或密码错误' };
    }
    return { success: true, user: { ...data.user, token: data.token } };
  } catch (e) {
    console.error('云端登录失败:', e);
    const isNetworkError = !navigator.onLine ||
      (e.message && (e.message.includes('Failed to fetch') ||
        e.message.includes('NetworkError') ||
        e.message.includes('network') ||
        e.message.includes('ERR_')));
    if (isNetworkError) {
      console.log('[auth] 网络不可用, 尝试离线登录缓存...');
      const offlineResult = await tryOfflineLogin(username, password);
      if (offlineResult.success) {
        return { success: true, user: offlineResult.user, offline: true };
      }
      return { success: false, error: '网络不可用, ' + (offlineResult.error || '离线登录失败') };
    }
    return { success: false, error: '登录失败: ' + (e.message || '网络错误') };
  }
}

};

const LoginLockout = {
  _getStorage() {
    if (typeof global.Capacitor !== 'undefined' && global.Capacitor.Plugins &&
    global.Capacitor.Plugins.Preferences) {
      return null; // APP 端暂不支持锁定, 仅桌面/网页版支持
    }
    try { return global.localStorage; } catch (e) { return null; }
  },
  checkLocked(username) {
    const storage = this._getStorage();
    if (!storage) return null;
    const lockUntil = parseInt(storage.getItem('auth:lockUntil:' + username) || '0', 10);
    if (lockUntil > Date.now()) {
      const remainMin = Math.ceil((lockUntil - Date.now()) / 60000);
      return '账号已被锁定, 请 ' + remainMin + ' 分钟后重试';
    }
    return null;
  },
  recordFailure(username) {
    const storage = this._getStorage();
    if (!storage) return '密码错误';
    const failKey = 'auth:failCount:' + username;
    let failCount = parseInt(storage.getItem(failKey) || '0', 10) + 1;
    if (failCount >= 5) {
      storage.setItem('auth:lockUntil:' + username, String(Date.now() + 30 * 60 * 1000));
      storage.removeItem(failKey);
      return '密码错误次数过多, 账号已被锁定 30 分钟';
    }
    storage.setItem(failKey, String(failCount));
    return '密码错误 (剩余 ' + (5 - failCount) + ' 次尝试机会)';
  },
  recordSuccess(username) {
    const storage = this._getStorage();
    if (!storage) return;
    storage.removeItem('auth:failCount:' + username);
  }
};

```

```

        storage.removeItem('auth:lockUntil:' + username);
    }
};
const AuditLog = {
    _getStorage() {
        if (typeof global.Capacitor !== 'undefined' && global.Capacitor.Plugins &&
global.Capacitor.Plugins.Preferences) {
            return null; // APP 端暂不支持，仅桌面/网页版
        }
        try { return global.localStorage; } catch (e) { return null; }
    },
    _resolveUser() {
        try {
            if (typeof currentUser !== 'undefined' && currentUser && currentUser.username) {
                return currentUser.username;
            }
        } catch (e) { }
        const storage = this._getStorage();
        if (storage) {
            try {
                const stored = storage.getItem('auth:currentUser');
                if (stored) return (JSON.parse(stored).username) || 'unknown';
            } catch (e) {}
        }
        return 'unknown';
    },
    record(action, details) {
        const storage = this._getStorage();
        if (!storage) return;
        try {
            const entry = {
                t: Date.now(),
                ts: new Date().toISOString(),
                user: this._resolveUser(),
                action: String(action || '').slice(0, 64),
                details: String(details || '').slice(0, 500)
            };
            let logs = [];
            try { logs = JSON.parse(storage.getItem('audit:log') || '[]'); } catch (e) {}
            if (!Array.isArray(logs)) logs = [];
            logs.push(entry);
            if (logs.length > 500) logs = logs.slice(-500);
            storage.setItem('audit:log', JSON.stringify(logs));
        } catch (e) { console.warn('审计日志写入失败:', e); }
    },
    list(limit) {
        const storage = this._getStorage();
        if (!storage) return [];
        try {
            const logs = JSON.parse(storage.getItem('audit:log') || '[]');
            if (!Array.isArray(logs)) return [];
            return limit ? logs.slice(-limit) : logs;
        } catch (e) { return []; }
    },
    clear() {
        const storage = this._getStorage();
        if (!storage) return;
        storage.removeItem('audit:log');
    }
};

```

```

    }
  };
  global.AuditLog = AuditLog;
  function createLocalAdapter(getUsersFn) {
    return {
      async authenticate(username, password) {
        try {
          const lockMsg = LoginLockout.checkLocked(username);
          if (lockMsg) return { success: false, error: lockMsg };
          const users = typeof getUsersFn === 'function' ? await getUsersFn() : getUsersFn;
          if (!Array.isArray(users)) {
            return { success: false, error: '用户数据加载失败' };
          }
          let user = users.find(u => u.username === username);
          if (!user) {
            user = users.find(u => u.phone === username);
          }
          if (!user) {
            return { success: false, error: '用户不存在' };
          }
          const pwdOk = await verifyPassword(password, user.password || '');
          if (!pwdOk) {
            return { success: false, error: LoginLockout.recordFailure(username) };
          }
          LoginLockout.recordSuccess(username);
          const { password: _, ...safeUser } = user;
          return { success: true, user: safeUser };
        } catch (e) {
          console.error('离线登录失败:', e);
          return { success: false, error: '登录失败: ' + (e.message || '未知错误') };
        }
      }
    };
  };
}

function createSingleUserAdapter(getUserFn) {
  return {
    async authenticate(username, password) {
      try {
        const lockMsg = LoginLockout.checkLocked(username);
        if (lockMsg) return { success: false, error: lockMsg };
        const user = typeof getUserFn === 'function' ? await getUserFn() : getUserFn;
        if (!user) {
          return { success: false, error: '用户信息加载失败' };
        }
        const usernameMatch = (user.username === username) || (user.phone === username);
        if (!usernameMatch) {
          return { success: false, error: '用户不存在' };
        }
        const pwdOk = await verifyPassword(password, user.password || '');
        if (!pwdOk) {
          return { success: false, error: LoginLockout.recordFailure(username) };
        }
        LoginLockout.recordSuccess(username);
        const { password: _, ...safeUser } = user;
        return { success: true, user: safeUser };
      } catch (e) {
        console.error('单用户登录失败:', e);
        return { success: false, error: '登录失败: ' + (e.message || '未知错误') };
      }
    }
  };
}

```

```

    }
  }
};
}
async function login(username, password, options = {}) {
  try {
    if (!username || !password) {
      return { success: false, error: '请输入手机号或用户名和密码' };
    }
    const adapter = options.adapter || cloudAdapter;
    const result = await adapter.authenticate(username, password);
    if (!result.success || !result.user) {
      try { AuditLog.record('login_failure', username + ': ' + (result.error || '认证失败')); } catch(e) {}

      return { success: false, error: result.error || '认证失败' };
    }
    const user = result.user;
    if (!result.offline) {
      await cacheOfflineLogin(username, password, user);
    }
    user.allowedMode = resolveAllowedMode(user);
    if (user.allowedMode === 'local' && !global.Capacitor) {
      user.allowedMode = 'both';
    }
    const userData = JSON.stringify(user);
    await StorageAdapter.setItem('auth:currentUser', userData);
    await StorageAdapter.setItem('auth:isLoggedIn', 'true');
    StorageAdapter.setSessionItem('auth:currentUser', userData);
    StorageAdapter.setSessionItem('auth:isLoggedIn', 'true');
    await StorageAdapter.setItem('auth:loginData', JSON.stringify({
      loginTime: Date.now(),
      username: user.username
    }));
    if (user.clinicName) {
      await StorageAdapter.setItem('auth:clinicName', user.clinicName);
    }
    startSessionMonitor(options.onSessionTimeout || null);
    return { success: true, user };
  } catch (e) {
    console.error('登录异常:', e);
    return { success: false, error: '登录失败: ' + (e.message || '未知错误') };
  }
}
}
async function logout() {
  stopSessionMonitor();
  try { AuditLog.record('logout', ''); } catch(e) {}
  const allKeys = [
    'auth:currentUser', 'auth:isLoggedIn', 'auth:loginData',
    'cloud_currentUser', 'cloud_isLoggedIn',
    'currentUser', 'isLoggedIn',
    'user_login_data',
    'cloud_prescription_cache', 'cloud_prescription_cache_time',
    'auth:offlineLoginCache',
    'auth:savedPassword'
  ];
  for (const key of allKeys) {
    await StorageAdapter.removeItem(key);
    StorageAdapter.removeSessionItem(key);
  }
}

```

```

    }
  }
  async function cacheOfflineLogin(username, password, user) {
    try {
      if (!username || !password || !user) return false;
      const encryptedPwd = await encryptPassword(password);
      if (!encryptedPwd) {
        console.warn('[auth] 离线缓存：密码加密失败，跳过缓存');
        return false;
      }
      const cacheData = {
        username: username,
        encryptedPassword: encryptedPwd,
        user: user,
        cachedAt: Date.now()
      };
      await StorageAdapter.setItem('auth:offlineLoginCache', JSON.stringify(cacheData));
      console.log('[auth] 离线登录缓存已保存');
      return true;
    } catch (e) {
      console.warn('[auth] 缓存离线登录失败:', e);
      return false;
    }
  }
}

async function tryOfflineLogin(username, password) {
  try {
    if (!username || !password) {
      return { success: false, error: '请输入手机号或用户名和密码' };
    }
    const cached = await StorageAdapter.getItem('auth:offlineLoginCache');
    if (!cached) {
      return { success: false, error: '无离线登录缓存，请联网登录' };
    }
    const cacheData = JSON.parse(cached);
    if (cacheData.username !== username) {
      return { success: false, error: '该用户无离线缓存，请联网登录' };
    }
    const CACHE_TTL = 30 * 24 * 60 * 60 * 1000;
    if (Date.now() - cacheData.cachedAt > CACHE_TTL) {
      await StorageAdapter.removeItem('auth:offlineLoginCache');
      return { success: false, error: '离线登录缓存已过期，请联网登录' };
    }
    const decryptedPwd = await decryptPassword(cacheData.encryptedPassword);
    if (!decryptedPwd) {
      await StorageAdapter.removeItem('auth:offlineLoginCache');
      return { success: false, error: '缓存密码解密失败，请联网登录' };
    }
    if (decryptedPwd !== password) {
      return { success: false, error: '密码错误' };
    }
    console.log('[auth] 离线登录成功（使用缓存凭证）');
    return { success: true, user: cacheData.user, offline: true };
  } catch (e) {
    console.warn('[auth] 离线登录失败:', e);
    return { success: false, error: '离线登录异常: ' + (e.message || '未知错误') };
  }
}

async function clearOfflineLoginCache() {

```

```

    try {
      await StorageAdapter.removeItem('auth:offlineLoginCache');
      console.log('[auth] 离线登录缓存已清除');
    } catch (e) {}
  }
}

async function saveRememberedUser(username) {
  const cleanUsername = String(username).trim();
  if (!cleanUsername) return;
  let remembered = [];
  try {
    const stored = await StorageAdapter.getItem('auth:rememberedUsers');
    if (stored) remembered = JSON.parse(stored);
    if (!Array.isArray(remembered)) remembered = [];
  } catch (e) { remembered = []; }
  remembered = remembered.filter(u => String(u).toLowerCase() !== cleanUsername.toLowerCase());
  remembered.unshift(cleanUsername);
  if (remembered.length > 5) remembered = remembered.slice(0, 5);
  await StorageAdapter.setItem('auth:rememberedUsers', JSON.stringify(remembered));
  await StorageAdapter.setItem('auth:rememberedUsername', cleanUsername);
}

async function loadRememberedUsers() {
  try {
    const [stored, single, cloudOld, localOld, legacyOld] = await Promise.all([
      StorageAdapter.getItem('auth:rememberedUsers'),
      StorageAdapter.getItem('auth:rememberedUsername'),
      StorageAdapter.getItem('cloud_rememberedUsername'),
      StorageAdapter.getItem('local_rememberedUsername'),
      StorageAdapter.getItem('rememberedUsername')
    ]);
    if (stored) {
      try {
        const arr = JSON.parse(stored);
        if (Array.isArray(arr) && arr.length > 0) return arr;
      } catch (e) {}
    }
    const oldSingle = single || cloudOld || localOld || legacyOld;
    return oldSingle ? [oldSingle] : [];
  } catch (e) {}
  return [];
}

async function clearRememberedUsers() {
  await StorageAdapter.removeItem('auth:rememberedUsers');
  await StorageAdapter.removeItem('auth:rememberedUsername');
  await StorageAdapter.removeItem('cloud_rememberedUsers');
  await StorageAdapter.removeItem('cloud_rememberedUsername');
  await StorageAdapter.removeItem('local_rememberedUsername');
  await StorageAdapter.removeItem('rememberedUsername');
  await StorageAdapter.removeItem('rememberedUsers');
}

migrateOldKeys().catch(e => console.warn('Key迁移失败:', e));
(async function initMasterKeyFromLicense() {
  try {
    if (global.electronAPI && global.electronAPI.license && typeof global.electronAPI.license.getStatus ===
'function') {
      const status = await global.electronAPI.license.getStatus();
      if (status && status.masterKey) {
        setMasterKey(status.masterKey);
        console.log('[AuthCore] masterKey 已从 license 注入');
      }
    }
  }
}

```

```
    }
  }
} catch (e) {
  console.warn('[AuthCore] 获取 masterKey 失败，使用硬编码盐 fallback:', e.message);
}
})();
global.AuthCore = {
  PASSWORD_SALT,
  SESSION_TIMEOUT_MS,
  CLOUD_API_BASE,
  StorageAdapter,
  SafeStorageBridge,
  hashPassword,
  hashPasswordWithUser,
  verifyPassword,
  isPasswordHashed,
  encryptUsers,
  decryptUsers,
  encryptPassword,
  decryptPassword,
  saveRememberedPassword,
  getRememberedPassword,
  validateUsername,
  validateAdminUsername,
  clearRememberedPassword,
  cacheOfflineLogin,
  tryOfflineLogin,
  clearOfflineLoginCache,
  resolveAllowedMode,
  isAdmin,
  isClinicAdmin,
  isPlatformAdmin,
  buildAuthPayload,
  checkSession,
  startSessionMonitor,
  stopSessionMonitor,
  login,
  logout,
  cloudAdapter,
  createLocalAdapter,
  createSingleUserAdapter,
  findUserByIdentifier,
  saveRememberedUser,
  loadRememberedUsers,
  clearRememberedUsers,
  migrateOldKeys,
  setMasterKey,
  validatePasswordStrength(password) {
    const result = { score: 0, label: '', errors: [] };
    if (!password) { result.label = '空'; return result; }
    if (password.length >= 8) result.score++; else result.errors.push('密码至少8位');
    if (password.length >= 12) result.score++;
    if (/[a-z]/.test(password)) result.score++; else result.errors.push('密码需包含小写字母');
    if (/[A-Z]/.test(password)) result.score++;
    if (/[\d]/.test(password)) result.score++; else result.errors.push('密码需包含数字');
    if (/^[a-zA-Z0-9]/.test(password)) result.score++;
    if (result.score <= 1) result.label = '太弱';
    else if (result.score <= 2) result.label = '弱';
  }
}
```

```

    else if (result.score <= 3) result.label = '一般';
    else if (result.score <= 4) result.label = '中等';
    else if (result.score <= 5) result.label = '强';
    else result.label = '非常强';
    return result;
  },
  async registerClinic(params) {
    const { clinicName, adminUsername, adminPassword, adminName, wechat } = params || {};
    if (!clinicName || !adminUsername || !adminPassword) {
      return { success: false, error: '请填写完整的注册信息' };
    }
    const usernameCheck = validateAdminUsername(adminUsername);
    if (!usernameCheck.valid) {
      return { success: false, error: usernameCheck.error };
    }
    const strength = this.validatePasswordStrength(adminPassword);
    if (strength.errors.length > 0) {
      return { success: false, error: strength.errors[0] };
    }
    try {
      const fetchFn = global.cloudFetch || global.fetch;
      const response = await fetchFn(CLOUD_API_BASE + '/users?action=register-clinic', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({
          clinicName: clinicName.trim(),
          adminUsername: adminUsername.trim(),
          adminPassword,
          adminName: (adminName || '').trim(),
          wechat: (wechat || '').trim()
        })
      });
      const data = (response && typeof response.json() === 'function')
        ? await response.json()
        : response;
      return data;
    } catch (e) {
      return { success: false, error: '注册请求失败: ' + (e.message || '网络错误') };
    }
  },
  validateActivationCode(code) {
    if (!code || typeof code !== 'string') {
      return { valid: false, error: '激活码不能为空' };
    }
    const trimmed = code.trim();
    const pattern = /^BNZC-[A-HJ-NP-Z2-9]{4}-[A-HJ-NP-Z2-9]{4}-[A-HJ-NP-Z2-9]{4}-[A-HJ-NP-Z2-9]{4}$/;
    if (!pattern.test(trimmed)) {
      return {
        valid: false,
        error: '激活码格式不正确',
        format: 'BNZC-XXXX-XXXX-XXXX-XXXX',
        note: 'X 为大写字母或数字（不含 I/O/0/1）'
      };
    }
    return { valid: true, code: trimmed };
  }
};
})(typeof window !== 'undefined' ? window : typeof globalThis !== 'undefined' ? globalThis : this);

```

```

(function (global) {
  'use strict';
  if (global.__licenseCheckInitialized) return;
  global.__licenseCheckInitialized = true;
  global.__licenseExpired = false;      // License 是否已失效
  global.__licenseActivating = false;   // 是否正在激活流程中（防止重复弹窗）
  let lastFailMessage = '授权已失效，请激活';
  async function performHeartbeatCheck() {
    try {
      const HEARTBEAT_INTERVAL_MS = 24 * 60 * 60 * 1000; // 24 小时
      const OFFLINE_LOCK_MS = 7 * 24 * 60 * 60 * 1000; // 7 天
      const now = Date.now();
      const lastHeartbeat = await StorageAdapter.getItem('license:lastHeartbeat');
      if (lastHeartbeat) {
        const lastTime = parseInt(lastHeartbeat, 10);
        if (now - lastTime < HEARTBEAT_INTERVAL_MS) {
          return; // 24 小时内已心跳，跳过
        }
      }
      const licenseCode = await StorageAdapter.getItem('license:code');
      const machineId = await StorageAdapter.getItem('license:machineId');
      if (!licenseCode || !machineId) {
        console.log('[Heartbeat] 无 license code 或 machineId，跳过心跳');
        return;
      }
      console.log('[Heartbeat] 开始心跳验证...');
      const response = await fetch('https://tcm-prescription-system.pages.dev/api/license/heartbeat', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ code: licenseCode, machineId: machineId })
      });
      if (!response.ok) {
        console.warn('[Heartbeat] 网络错误，HTTP', response.status);
        const offlineStart = await StorageAdapter.getItem('license:offlineStart');
        if (!offlineStart) {
          await StorageAdapter.setItem('license:offlineStart', String(now));
        }
        if (offlineStart) {
          const offlineTime = now - parseInt(offlineStart, 10);
          if (offlineTime > OFFLINE_LOCK_MS) {
            console.error('[Heartbeat] 离线超过 7 天，锁定应用');
            global.__licenseExpired = true;
            await showExpireAlertAndActivate('应用已离线超过 7 天，请联网验证后继续使用');
          }
        }
      }
      return;
    }
    const data = await response.json();
    if (data.success && data.valid && data.action === 'ok') {
      await StorageAdapter.setItem('license:lastHeartbeat', String(now));
      await StorageAdapter.removeItem('license:offlineStart');
      console.log('[Heartbeat] 心跳成功，剩余天数:', data.daysRemaining);
    } else {
      console.error('[Heartbeat] 心跳失败:', data.action);
      global.__licenseExpired = true;
      const msg = {
        'expired': '授权已过期，请续费后激活',
        'disabled': '授权已被禁用，请联系客服',
      }
    }
  }
}

```

```

        'unknown': '授权信息无效, 请重新激活',
        'device_mismatch': '设备不匹配, 请在当前设备重新激活'
    }[data.action] || '授权验证失败, 请重新激活';
    await showExpireAlertAndActivate(msg);
  }
} catch (e) {
  console.warn('[Heartbeat] 异常:', e.message);
  const now = Date.now();
  const offlineStart = await StorageAdapter.getItem('license:offlineStart');
  if (!offlineStart) {
    await StorageAdapter.setItem('license:offlineStart', String(now));
  }
}
}
}
async function enforceTrialPersonalEdition() {
  try {
    if (!global.electronAPI || !global.electronAPI.license ||
      typeof global.electronAPI.license.getStatus !== 'function') {
      return false;
    }
    const status = await global.electronAPI.license.getStatus();
    const licenseType = (status && (status.licenseType || status.type)) || '';
    if (licenseType !== 'trial') return false;
    if (typeof CONFIG !== 'undefined' && CONFIG && CONFIG.edition !== 'personal') {
      CONFIG.edition = 'personal';
      console.log('[Trial] 试用期校正 CONFIG.edition -> personal');
    }
    renderTrialStandardVersion();
    return true;
  } catch (e) {
    console.warn('[Trial] 标准版校正异常 (非致命):', e.message);
    return false;
  }
}
}
function renderTrialStandardVersion() {
  try {
    const label = '【离线标准版】 V1.0.0';
    const tag = document.querySelector('.version-tag');
    if (tag && tag.textContent && tag.textContent.indexOf('标准版') < 0) {
      tag.textContent = label;
    }
    const hints = document.querySelectorAll('.tab-hint');
    hints.forEach(function(h) {
      const spans = h.querySelectorAll('span');
      spans.forEach(function(s) {
        if (s.textContent && (s.textContent.indexOf('本地') >= 0 || s.textContent.indexOf('离线') >=
0)) {
          s.textContent = label;
        }
      });
    });
    if (typeof global.updateUserDisplay === 'function') global.updateUserDisplay();
    if (typeof global.updateMobileActionButtons === 'function') global.updateMobileActionButtons();
  } catch (e) {
    console.warn('[Trial] 标准版界面刷新失败:', e);
  }
}
}
async function checkLicenseAndShowActivate() {

```

```

try {
  if (!global.electronAPI || !global.electronAPI.license ||
    typeof global.electronAPI.license.validate !== 'function') {
    console.log('[LicenseCheck] 未检测到 license API, 跳过校验');
    return;
  }
  const result = await global.electronAPI.license.validate();
  if (result && result.valid) {
    console.log('[LicenseCheck] 授权有效:', result.message || '');
    global.__licenseExpired = false;
    global.__licenseActivating = false;
    await enforceTrialPersonalEdition();
    performHeartbeatCheck();
    if (result.needOnlineVerify && global.electronAPI.license.verifyOnline) {
      try {
        console.log('[LicenseCheck] 检测到需要在线验证, 正在验证...');
        const verifyResult = await global.electronAPI.license.verifyOnline();
        if (verifyResult && verifyResult.success) {
          console.log('[LicenseCheck] 在线验证成功');
        } else {
          console.warn('[LicenseCheck] 在线验证失败:', verifyResult && verifyResult.error);
        }
      } catch (e) {
        console.warn('[LicenseCheck] 在线验证异常:', e);
      }
    }
    return;
  }
  const msg = (result && result.message) ? result.message : '授权已失效, 请激活';
  lastFailMessage = msg;
  console.warn('[LicenseCheck] 授权失效:', msg);
  global.__licenseExpired = true;
  if (global.__licenseActivating) {
    console.log('[LicenseCheck] 激活流程进行中, 跳过弹窗');
    return;
  }
  await showExpireAlertAndActivate(msg);
} catch (e) {
  console.error('[LicenseCheck] 校验异常:', e);
  global.__licenseActivating = false;
}
}

async function showExpireAlertAndActivate(msg) {
  global.__licenseActivating = true;
  if (global.electronAPI && global.electronAPI.activate &&
    typeof global.electronAPI.activate.showExpireAlert === 'function') {
    try {
      await global.electronAPI.activate.showExpireAlert(msg);
      return;
    } catch (e) {
      console.error('[LicenseCheck] showExpireAlert 失败, 回退到 alert+show:', e);
    }
  }
  try {
    alert(msg);
  } catch (e) { }
  if (global.electronAPI && global.electronAPI.activate &&
    typeof global.electronAPI.activate.show === 'function') {

```

```

    try {
      await global.electronAPI.activate.show();
    } catch (e) {
      console.error('[LicenseCheck] 拉起激活窗口失败:', e);
      global.__licenseActivating = false;
    }
  } else {
    try { alert(msg + '\n\n请联系管理员获取激活码'); } catch (e) {}
    global.__licenseActivating = false;
  }
}

let fallbackTimer = null;
function startFallbackCheck() {
  if (fallbackTimer) clearInterval(fallbackTimer);
  fallbackTimer = setInterval(async () => {
    if (!global.__licenseExpired || global.__licenseActivating) return;
    console.log('[LicenseCheck] 兜底检查: license 失效, 重新弹激活窗口');
    await showExpireAlertAndActivate(lastFailMessage);
  }, 5000); // 每 5 秒检查一次
}

async function showActivateDialog() {
  try {
    if (!global.electronAPI || !global.electronAPI.activate) {
      showHtmlAlert('授权系统未就绪, 请重启应用后重试');
      global.__licenseActivating = false;
      return;
    }
    let machineId = '';
    try {
      const r = await global.electronAPI.activate.getMachineId();
      machineId = (r && r.machineId) ? r.machineId : (r || '');
    } catch (e) {}
    let clinicName = '';
    try {
      if (typeof CONFIG !== 'undefined' && CONFIG.clinicName) {
        clinicName = CONFIG.clinicName;
      }
    } catch (e) {}
    const modalResult = await showActivateModal(machineId, clinicName);
    if (modalResult.cancelled || !modalResult.code || !modalResult.code.trim()) {
      global.__licenseActivating = false;
      if (modalResult.trial) {
        await enforceTrialPersonalEdition();
        await showHtmlAlert('✅ 已进入试用模式 (免费7天 · 标准版) \n\n试用期内可正常使用, 到期后请在激活窗口输入激活码激活正式授权。');
      } else {
        console.log('[LicenseCheck] 用户取消激活');
      }
    }
    return;
  }
  let user = '';
  try {
    if (typeof CONFIG !== 'undefined' && CONFIG.doctorName) {
      user = CONFIG.doctorName;
    } else {
      user = localStorage.getItem('auth:rememberedUsername') || '';
    }
  } catch (e) {}
}

```

```

const codeTrim = modalResult.code.trim();
const codePattern = /^BNZC-[A-HJ-NP-Z2-9]{4}-[A-HJ-NP-Z2-9]{4}-[A-HJ-NP-Z2-9]{4}-[A-HJ-NP-Z2-9]{4}$/;
if (!codePattern.test(codeTrim)) {
  await showHtmlAlert('激活码格式不正确\n\n正确格式：BNZC-XXXX-XXXX-XXXX-XXXX\n (X
为大写字母或数字，去除 I/O/0/1) \n\n点击确定重新输入');
  global.__licenseActivating = false;
  showActivateDialog();
  return;
}
const result = await global.electronAPI.activate.submit(codeTrim, user);
if (result && result.success) {
  global.__licenseExpired = false;
  global.__licenseActivating = false;
  try {
    await StorageAdapter.setItem('license:code', codeTrim);
    if (global.electronAPI && global.electronAPI.license &&
      typeof global.electronAPI.license.getMachineId === 'function') {
      const mid = await global.electronAPI.license.getMachineId();
      if (mid) await StorageAdapter.setItem('license:machineId', String(mid));
    }
    await StorageAdapter.removeItem('license:lastHeartbeat');
    await StorageAdapter.removeItem('license:offlineStart');
  } catch(e) { console.warn('[Heartbeat] 存储 license code 失败:', e); }
  showHtmlAlert('✅ 激活成功! \n' + (result.message || '') + '\n\n点击确定后应用将重启');
  global.electronAPI.activate.restart();
} else {
  const errMsg = (result && result.error) ? result.error : '未知错误';
  const friendlyMsg = formatActivateError(errMsg);
  await showHtmlAlert(friendlyMsg);
  global.__licenseActivating = false;
  showActivateDialog();
}
} catch (e) {
  await showHtmlAlert('激活过程出错: ' + e.message);
  global.__licenseActivating = false;
}
}

function formatActivateError(errMsg) {
  const msg = String(errMsg || '').toLowerCase();
  if (msg.includes('网络') || msg.includes('超时') || msg.includes('timeout') ||
    msg.includes('unknownhost') || msg.includes('socket') || msg.includes('连接')) {
    return '❌ 激活失败：网络连接异常\n\n' + errMsg +
'\n\n请检查网络后重试。如暂时无法联网，请联系客服协助激活。 \n\n点击确定重新输入激活码';
  }
  if (msg.includes('激活码') || msg.includes('已禁用') || msg.includes('已过期') || msg.includes('disabled')
|| msg.includes('expired')) {
    return '❌ 激活失败：激活码无效\n\n' + errMsg +
'\n\n请确认激活码输入正确，或联系客服重新获取。 \n\n点击确定重新输入激活码';
  }
  if (msg.includes('诊所') || msg.includes('绑定') || msg.includes('clinic') || msg.includes('绑定设备')) {
    return '❌ 激活失败：授权绑定不匹配\n\n' + errMsg +
'\n\n请确认诊所名称和机器ID与客服记录一致。 \n\n点击确定重新输入激活码';
  }
  if (msg.includes('限速') || msg.includes('频繁') || msg.includes('rate')) {
    return '❌ 激活失败：尝试过于频繁\n\n' + errMsg + '\n\n请等待几分钟后重试。 \n\n点击确定重新输入激活码';
  }
  return '❌ 激活失败: \n' + errMsg + '\n\n点击确定重新输入激活码';
}
}

```

```

function copyTextToClipboard(text) {
  return new Promise(function(resolve) {
    const textStr = String(text || '');
    if (!textStr) { resolve(false); return; }
    if (navigator.clipboard && navigator.clipboard.writeText) {
      navigator.clipboard.writeText(textStr).then(function() {
        resolve(true);
      }).catch(function() {
        fallbackCopy(textStr, resolve);
      });
    } else {
      fallbackCopy(textStr, resolve);
    }
  });
}

function fallbackCopy(text, resolve) {
  try {
    const ta = document.createElement('textarea');
    ta.value = text;
    ta.style.cssText = 'position:fixed;left:-9999px;top:-9999px;opacity:0;';
    document.body.appendChild(ta);
    ta.focus();
    ta.select();
    const ok = document.execCommand('copy');
    document.body.removeChild(ta);
    resolve(ok);
  } catch (e) {
    resolve(false);
  }
}

function showActivateModal(machineId, clinicName) {
  return new Promise(function(resolve) {
    const overlay = document.createElement('div');
    overlay.id = 'activateModalOverlay';
    overlay.style.cssText =
      'position:fixed;top:0;left:0;width:100%;height:100%;background:rgba(0,0,0,0.6);z-index:99999;display
: flex;align-items:center;justify-content:center;padding:16px;';
    const card = document.createElement('div');
    card.style.cssText =
      'background:white;border-radius:14px;width:100%;max-width:400px;padding:20px;box-shadow:0 10px 30px
      rgba(0,0,0,0.3);max-height:90vh;overflow-y:auto;';
    card.innerHTML =
      '<div style="background:linear-gradient(135deg,#667eea 0%,#764ba2 100%);margin:-20px -20px 16px
      -20px;padding:20px;border-radius:14px 14px 0 0;text-align:center;">' +
        '<div style="font-size:20px;font-weight:bold;color:white;">🔒 软件激活</div>' +
        '<div style="font-size:12px;color:rgba(255,255,255,0.85);margin-top:4px;">惠康中医诊所管理系统</
      div>' +
        '</div>' +
        '<div style="background:#f5f7ff;border-radius:8px;padding:12px;margin-bottom:16px;">' +
          '<div style="font-size:13px;font-weight:bold;color:#555;margin-bottom:6px;">📖 激活步骤</div>' +
          '<div style="font-size:12px;color:#666;line-height:1.8;">' +
            '<div>❶ 点击下方"复制全部信息", 复制设备识别码和诊所名</div>' +
            '<div>❷ 前往官网"购买激活码"页面生成订单号, 或直接联系客服</div>' +
            '<div>❸ 将信息发给客服, 付款后获取激活码</div>' +
            '<div>❹ 在此页面输入激活码, 点击"立即激活"</div>' +
          '</div>' +
        '</div>' +
        '<div style="margin-bottom:14px;">'

```

```

'<div style="display:flex;align-items:center;justify-content:space-between;margin-bottom:6px;">'
+
    '<span style="font-size:12px;color:#888;font-weight:bold;">🔑 设备识别码</span>' +
    '<div style="display:flex;gap:6px;">' +
        '<button id="copyMachineIdBtn" style="font-size:11px;padding:4px 10px;border:1px solid #667eea;border-radius:4px;background:white;color:#667eea;cursor:pointer;">复制ID</button>' +
        '<button id="copyAllInfoBtn" style="font-size:11px;padding:4px 10px;border:1px solid #667eea;border-radius:4px;background:#667eea;color:white;cursor:pointer;font-weight:bold;">📄 复制全部信息</button>' +
    '</div>' +
    '</div>' +
    '<div id="machineIdValue" style="font-size:11px;color:#333;background:#f9f9f9;border:1px solid #eee;border-radius:6px;padding:8px 10px;word-break:break-all;font-family:monospace;letter-spacing:1px;">' +
        (machineId || '未知') + '</div>' +
    '</div>' +
    (clinicName ?
        '<div style="margin-bottom:14px;font-size:12px;color:#666;">' +
            '<span style="color:#888;font-weight:bold;">🏢 本机诊所名: </span>' +
            '<span style="color:#333;">' + clinicName + '</span>' +
        '</div>' : '') +
    '<div style="font-size:12px;color:#666;margin-bottom:6px;">激活码格式: BNZC-XXXX-XXXX-XXXX-XXXX</div>' +
    '>' +
        '<input type="text" id="activateCodeInput" ' +
        'style="width:100%;padding:14px 16px;font-size:16px;font-family:monospace;border:2px solid #ddd;border-radius:8px;letter-spacing:2px;outline:none;transition:border-color 0.2s;margin-bottom:14px;box-sizing:border-box;" ' +
        'placeholder="请输入激活码" autocomplete="new-password" data-lpignore="true" />' +
        '<div id="activateLoadingBox" style="display:none;text-align:center;padding:10px;margin-bottom:14px;' +
    ">' +
        '<div style="display:inline-block;width:20px;height:20px;border:2px solid #ddd;border-top-color:#667eea;border-radius:50%;animation:activateSpin 0.8s linear infinite;vertical-align:middle;margin-right:8px;"></div>' +
        '<span style="font-size:13px;color:#667eea;vertical-align:middle;">正在激活, 请稍候...</span>' +
    '</div>' +
        '<div style="background:#fff8e1;border-radius:8px;padding:10px;margin-bottom:16px;">' +
        '<div style="font-size:12px;font-weight:bold;color:#e65100;margin-bottom:6px;">🔑 获取激活码</div>' +
        '<a href="https://tcm-prescription-system.pages.dev/download" target="_blank" style="display:block;text-align:center;padding:10px;margin-bottom:8px;background:linear-gradient(135deg,#667eea 0%,#764ba2 100%);color:white;text-decoration:none;border-radius:8px;font-size:13px;font-weight:bold;">🌐 前往官网购买激活码</a>' +
        '<div style="font-size:11px;color:#999;text-align:center;margin-bottom:8px;">官网"购买激活码"Tab 可一键生成订单信息</div>' +
        '<div style="border-top:1px dashed #ffe082;padding-top:8px;">' +
            '<div style="font-size:12px;color:#555;line-height:1.8;">' +
                '<div style="display:flex;align-items:center;justify-content:space-between;">' +
                    '<span>客服微信: <strong style="color:#333;">hktzy1688</strong></span>' +
                    '<button class="copyContactBtn" data-text="hktzy1688" style="font-size:11px;padding:2px 8px;border:1px solid #e65100;border-radius:4px;background:white;color:#e65100;cursor:pointer;">复制</button>' +
                '</div>' +
            '</div>' +
        '</div>' +
    '</div>' +
    '<div style="display:flex;gap:10px;">' +
        '<button id="activateCancelBtn" style="flex:1;padding:12px;font-size:15px;border:1px solid #ddd;border-radius:8px;color:#666;background:white;cursor:pointer;">取消</button>' +
        '<button id="activateSubmitBtn"

```

```

style="flex:1;padding:12px;font-size:15px;border:none;border-radius:8px;color:white;background:linear-gradient(135
deg,#667eea 0%,#764ba2 100%);cursor:pointer;font-weight:bold;">立即激活</button>' +
'</div>' +
'<button id="activateTrialBtn" style="width:100%;margin-top:10px;padding:12px;font-size:15px;border:
1px solid #4caf50;border-radius:8px;color:#4caf50;background:#f0f9eb;cursor:pointer;font-weight:bold;">⌚
立即试用（免费7天 · 标准版）</button>'
if (!document.getElementById('activateSpinKeyframes')) {
  const styleEl = document.createElement('style');
  styleEl.id = 'activateSpinKeyframes';
  styleEl.textContent = '@keyframes activateSpin{to{transform:rotate(360deg);}}';
  document.head.appendChild(styleEl);
}
overlay.appendChild(card);
document.body.appendChild(overlay);
const input = card.querySelector('#activateCodeInput');
const cancelBtn = card.querySelector('#activateCancelBtn');
const submitBtn = card.querySelector('#activateSubmitBtn');
const copyMachineIdBtn = card.querySelector('#copyMachineIdBtn');
const loadingBox = card.querySelector('#activateLoadingBox');
const copyContactBtns = card.querySelectorAll('.copyContactBtn');
const trialBtn = card.querySelector('#activateTrialBtn');
setTimeout(function() { input.focus(); }, 350);
copyMachineIdBtn.addEventListener('click', async function() {
  const ok = await copyTextToClipboard(machineId);
  copyMachineIdBtn.textContent = ok ? '✅ 已复制' : '❌ 失败';
  setTimeout(function() { copyMachineIdBtn.textContent = '复制ID'; }, 1500);
});
const copyAllInfoBtn = card.querySelector('#copyAllInfoBtn');
copyAllInfoBtn.addEventListener('click', async function() {
  const allInfo = '惠康中医激活信息\n' +
    '设备识别码: ' + (machineId || '未知') + '\n' +
    '诊所名: ' + (clinicName || '未设置') + '\n' +
    '请帮我生成激活码, 谢谢! ';
  const ok = await copyTextToClipboard(allInfo);
  copyAllInfoBtn.textContent = ok ? '✅ 已复制全部信息' : '❌ 失败';
  setTimeout(function() { copyAllInfoBtn.textContent = '📋 复制全部信息'; }, 2000);
});
copyContactBtns.forEach(function(btn) {
  btn.addEventListener('click', async function() {
    const text = btn.getAttribute('data-text') || '';
    const ok = await copyTextToClipboard(text);
    btn.textContent = ok ? '✅ 已复制' : '❌ 失败';
    setTimeout(function() { btn.textContent = '复制'; }, 1500);
  });
});
function cleanup() {
  if (overlay.parentNode) overlay.parentNode.removeChild(overlay);
}
function submitCode() {
  const val = input.value;
  if (val && val.trim()) {
    submitBtn.disabled = true;
    cancelBtn.disabled = true;
    input.disabled = true;
    loadingBox.style.display = 'block';
    submitBtn.textContent = '激活中...';
    setTimeout(function() {
      cleanup();
    }, 3000);
  }
}

```

```

    });
    r = await resp.json();
  } catch (err) { console.warn('[LicenseCheck] admin-status 网络错误:', err); }
  if (r && r.success && r.status === 'activated') {
    clearInterval(pollTimer); pollTimer = null;
    await onAdminActivated(r, requestId);
  } else if (r && r.success && r.status === 'rejected') {
    clearInterval(pollTimer); pollTimer = null;
    document.getElementById('adminRejectReason').textContent = r.reason || '未知原因';
    show('adminRejected');
  }
  if (count >= 120) {
    clearInterval(pollTimer); pollTimer = null;
    if (statusEl) statusEl.textContent = '🕒 等待时间较长，管理员可能还在处理\n关闭窗口不影响审核';
  }
}, 5000);
}
async function onAdminActivated(r, requestId) {
  const license = r.license || '';
  const phone = state.phone;
  const descEl = document.getElementById('adminSuccessDesc');
  document.getElementById('adminSuccessPhone').textContent = phone;
  try {
    if (typeof window.addLocalActivationUser === 'function') {
      window.addLocalActivationUser({
        username: phone || state.adminName || '',
        password: state.password || 'admin',
        name: state.adminName || phone || '管理员'
      });
    }
  } catch (ea) { console.warn('同步激活登录账号到前端失败:', ea); }
  if (global.electronAPI && global.electronAPI.activate &&
    typeof global.electronAPI.activate.installAdminLicense === 'function' && license) {
    try {
      const inst = await global.electronAPI.activate.installAdminLicense({
        license: license,
        adminName: state.adminName,
        clinicName: state.clinicName,
        password: state.password || 'admin',
        phone: phone
      });
      if (inst && inst.success) {
        descEl.innerHTML = '管理员已通过您的激活申请<br>软件即将重启，请使用手机号登录';
        show('adminSuccess');
        document.getElementById('adminSuccessBtn').textContent = '🔄 重启应用';
        document.getElementById('adminSuccessBtn').onclick = function() {
          if (global.electronAPI && global.electronAPI.activate &&
            global.electronAPI.activate.restart) {
            try { global.electronAPI.activate.restart(); } catch(e){}
          }
        };
      } else {
        descEl.innerHTML = '激活已通过，但本地写入失败: ' + ((inst && inst.error) || '未知错误') +
        '<br>请将机器ID发给客服人工激活';
        show('adminSuccess');
      }
    } catch (e) {
      descEl.innerHTML = '激活已通过，但本地写入失败: ' + (e && e.message || '未知错误');
    }
  }
}

```

```

        show('adminSuccess');
    }
} else {
    descEl.innerHTML = '管理员已通过您的激活申请<br>请返回登录框，使用手机号登录';
    show('adminSuccess');
    document.getElementById('adminSuccessBtn').textContent = '✅ 好的';
    document.getElementById('adminSuccessBtn').onclick = function() { cleanup(); };
}
}
document.getElementById('adminRetryBtn').addEventListener('click', function() { show('adminStepEdition');
});

document.getElementById('adminCopyMidBtn').addEventListener('click', async function() {
    const ok = await copyTextToClipboard(machineId || '');
    const b = document.getElementById('adminCopyMidBtn');
    b.textContent = ok ? '✅' : '❌';
    setTimeout(function(){ b.textContent = '复制'; }, 1200);
});
document.getElementById('adminCloseBtn').addEventListener('click', cleanup);
overlay.addEventListener('click', function(e) { if (e.target === overlay) cleanup(); });
if (clinicName) document.getElementById('adminClinicName').value = clinicName;
}
function startLicenseCheck() {
    setTimeout(async () => {
        await checkLicenseAndShowActivate();
        startFallbackCheck();
        injectLicenseStatusIntoSettings();
    }, 2000);
}
if (document.readyState === 'complete' || document.readyState === 'interactive') {
    startLicenseCheck();
} else {
    document.addEventListener('DOMContentLoaded', startLicenseCheck);
}
})(typeof window !== 'undefined' ? window : typeof globalThis !== 'undefined' ? globalThis : this);
(function (global) {
    'use strict';
    function setupKeyboardAdapter() {
        if (global.__bnKbAdapter) return;
        global.__bnKbAdapter = true;
        function scrollToActive() {
            var el = document.activeElement;
            if (!el) return;
            if (el.tagName === 'INPUT' || el.tagName === 'TEXTAREA' || el.isContentEditable) {
                try { el.scrollIntoView({ block: 'center', behavior: 'smooth' }); } catch (e) {
                    el.scrollIntoView(false);
                }
            }
        }
        document.addEventListener('focusin', function (e) {
            var el = e.target;
            if (el && (el.tagName === 'INPUT' || el.tagName === 'TEXTAREA' || el.isContentEditable)) {
                setTimeout(scrollToActive, 300);
            }
        });
        if (global.visualViewport) {
            var timer = null;
            global.visualViewport.addEventListener('resize', function () {
                if (timer) clearTimeout(timer);
                timer = setTimeout(scrollToActive, 100);
            });
        }
    }
}

```

```

    });
  }
}
if (document.readyState === 'complete' || document.readyState === 'interactive') {
  setupKeyboardAdapter();
} else {
  document.addEventListener('DOMContentLoaded', setupKeyboardAdapter);
}
})(typeof window !== 'undefined' ? window : typeof globalThis !== 'undefined' ? globalThis : this);
/* ===== 源文件: security-guard.js ===== */
(function (global) {
  'use strict';
  const SecurityGuard = {
    _enabled: true,
    _debuggerDetected: false,
    _integrityChecked: false,
    init() {
      try {
        var isLocalFile = location.protocol === 'file:' || location.protocol === 'capacitor:';
        var hasDebugParam = location.search.indexOf('debug') !== -1;
        if (isLocalFile && hasDebugParam) {
          this._enabled = false;
          return;
        }
      } catch (e) { }
      if (!this._enabled) return;
      this._startAntiDebug();
      if (document.readyState === 'complete' || document.readyState === 'interactive') {
        setTimeout(() => this._checkIntegrity(), 500);
      } else {
        document.addEventListener('DOMContentLoaded', () => {
          setTimeout(() => this._checkIntegrity(), 500);
        });
      }
    },
    _startAntiDebug() {
      const self = this;
      setInterval(() => {
        if (!self._enabled || self._debuggerDetected) return;
        const start = Date.now();
        try {
          (function () { }).constructor('debugger').call();
        } catch (e) { }
        const elapsed = Date.now() - start;
        if (elapsed > 100) {
          self._debuggerDetected = true;
          console.warn('[SecurityGuard] 检测到调试器附加 (时间差: ' + elapsed + 'ms)');
        }
      }, 5000);
      setInterval(() => {
        if (!self._enabled || self._debuggerDetected) return;
        try {
          const threshold = 160;
          const widthDiff = window.outerWidth - window.innerWidth;
          const heightDiff = window.outerHeight - window.innerHeight;
          if (widthDiff > threshold || heightDiff > threshold) {
            self._debuggerDetected = true;
            console.warn('[SecurityGuard] 检测到 DevTools 可能已打开');
          }
        }
      }, 5000);
    }
  };
})(global);

```

```

    }
    } catch (e) { }
  }, 10000);
},
_checkIntegrity() {
  if (!this._enabled || this._integrityChecked) return;
  try {
    const criticalElements = [
      'prescriptionPaper', // 处方签纸
      'prescriptionBody',  // 药物表体
      'historyList',        // 历史列表
      'clinicNameDisplay'   // 诊所名显示
    ];
    const missing = criticalElements.filter(id => !document.getElementById(id));
    if (missing.length > 0) {
      console.warn('[SecurityGuard] 完整性校验失败, 缺失关键元素:', missing);
    }
    const criticalFunctions = [
      'savePrescription',
      'loadData',
      'handleLogin'
    ];
    const missingFunctions = criticalFunctions.filter(fn => typeof global[fn] !== 'function');
    if (missingFunctions.length > 0) {
      console.warn('[SecurityGuard] 完整性校验失败, 缺失关键函数:', missingFunctions);
    }
    if (global.electronAPI && global.electronAPI.isElectron) {
      const criticalAPIs = ['getCurrentUser', 'saveUserData', 'getUserData'];
      const missingAPIs = criticalAPIs.filter(api => typeof global.electronAPI[api] !== 'function');
      if (missingAPIs.length > 0) {
        console.warn('[SecurityGuard] Electron API 校验失败, 缺失:', missingAPIs);
      }
    }
    this._integrityChecked = true;
  } catch (e) {
    console.warn('[SecurityGuard] 完整性校验异常:', e);
  }
},
isDebuggerDetected() {
  return this._debuggerDetected;
},
isEnabled() {
  return this._enabled;
}
};
if (typeof window !== 'undefined') {
  try {
    SecurityGuard.init();
  } catch (e) {
    console.warn('[SecurityGuard] 初始化失败:', e);
  }
}
global.SecurityGuard = SecurityGuard;
})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
/* ===== 源文件: permission.js ===== */
(function (global) {
  'use strict';
  const Permission = {

```

```

    _edition: null,
    _config: null,
    _initialized: false,
    async init() {
        if (this._initialized) return;
        this._initialized = true;
        try {
            if (global.electronAPI && global.electronAPI.getAppConfig) {
                const result = await global.electronAPI.getAppConfig();
                this._config = (result && result.config) ? result.config : (result || {});
            } else if (typeof CONFIG !== 'undefined' && CONFIG) {
                this._config = CONFIG;
            } else {
                const stored = localStorage.getItem('app_config');
                this._config = stored ? JSON.parse(stored) : {};
            }
        } catch (e) {
            console.warn('读取版本配置失败:', e);
            this._config = {};
        }
        this._edition = this._config.edition || (global.EDITION) || 'cloud';
        try {
            if (typeof CONFIG !== 'undefined' && CONFIG && CONFIG.edition !== this._edition) {
                CONFIG.edition = this._edition;
                console.log('[DBG] Permission synced CONFIG.edition ->', this._edition);
            }
        } catch (_) { }
        console.log('[DBG] Permission initialized, edition:', this._edition);
    },
    get edition() { return this._edition; },
    get config() { return this._config; },
    isCloud() {
        return ['cloud', 'cloud_personal', 'cloud_clinic'].includes(this._edition);
    },
    isOffline() {
        return ['offline', 'personal', 'clinic_custom', 'clinic',
            'offline_personal', 'offline_clinic'].includes(this._edition);
    },
    isPersonal() {
        return ['personal', 'cloud_personal', 'offline_personal'].includes(this._edition);
    },
    isInstitutional() {
        return ['clinic_custom', 'offline', 'clinic', 'cloud_clinic', 'offline_clinic',
            'cloud'].includes(this._edition);
    },
    isClinicCustom() {
        return this.isInstitutional();
    },
    canEditClinicName() {
        return true;
    },
    canEditDoctorName() {
        return true;
    },
    canManageUsers() {
        return this.isInstitutional();
    },
    canSync() {

```

```

        return this.isCloud();
    },
    hasMultiUser() {
        return this.isInstitutional();
    },
    hasRememberPassword() {
        return false;
    },
    hasUsernameDropdown() {
        return this.isInstitutional();
    },
    _isStandardEditionForced() {
        try {
            var INST_ED = ['clinic', 'offline_clinic', 'clinic_custom', 'offline', 'cloud_clinic', 'cloud'];
            try {
                var cfgInst = (typeof CONFIG !== 'undefined' && CONFIG && CONFIG.edition) ?
String(CONFIG.edition) : '';
                var winInst = String(global.EDITION || '');
                if (cfgInst && INST_ED.indexOf(cfgInst) >= 0) return false;
                if (winInst && INST_ED.indexOf(winInst) >= 0) return false;
                if (this._edition && INST_ED.indexOf(String(this._edition)) >= 0) return false;
            } catch (_) {}
            var cfgEd = (typeof CONFIG !== 'undefined' && CONFIG && CONFIG.edition) ? String(CONFIG.edition) :
'';

            var winEd = String(global.EDITION || '');
            if (['personal', 'offline_personal'].indexOf(cfgEd) >= 0) return true;
            if (['personal', 'offline_personal'].indexOf(winEd) >= 0) return true;
            var cfgProd = (typeof CONFIG !== 'undefined' && CONFIG && CONFIG.productName) ?
String(CONFIG.productName) : '';
            var winProd = String(global.PRODUCT_NAME || '');
            if (cfgProd === '惠康中医-本地' || winProd === '惠康中医-本地') return true;
        } try {
            if (global.document && document.getElementById &&
document.getElementById('_force_standard_edition_marker_')) return true;
        } catch (_) {}
        } catch (_) {}
        return false;
    },
    canManageUsersByRole(user) {
        if (this._isStandardEditionForced()) return false;
        if (!this.isInstitutional()) return false;
        if (!user) return false;
        if (global.AuthCore && global.AuthCore.isClinicAdmin) {
            return global.AuthCore.isClinicAdmin(user);
        }
        return user.role === 'admin' || user.role === 'clinic_admin';
    },
    canChangePassword(user) {
        if (this._isStandardEditionForced()) return true;
        if (!this.isInstitutional()) return true; // 标准版/单用户允许改密
        if (!user) return false;
        if (global.AuthCore && global.AuthCore.isClinicAdmin) {
            return !global.AuthCore.isClinicAdmin(user);
        }
        return user.role !== 'admin' && user.role !== 'clinic_admin';
    },
    shouldShowUserManage(user) {
        return this.canManageUsersByRole(user);
    }

```

```

    },
    shouldShowChangePwd(user) {
        return this.canChangePassword(user);
    },
    shouldShowAutoStart(user) {
        if (this.isPersonal()) return false;
        if (!user) return false;
        if (global.AuthCore && global.AuthCore.isClinicAdmin) {
            return global.AuthCore.isClinicAdmin(user);
        }
        return user.role === 'admin' || user.role === 'clinic_admin';
    },
    canViewAllPrescriptions(user) {
        if (!user) return false;
        if (global.AuthCore && global.AuthCore.isClinicAdmin) {
            return global.AuthCore.isClinicAdmin(user);
        }
        return user.role === 'admin' || user.role === 'clinic_admin';
    },
    applyRuntimePermissions() {
        const edition = this._edition;
        console.log('[DBG] Applying runtime permissions for edition:', edition);
        if (this.isPersonal()) {
            const userManageBtn = document.getElementById('userManageBtn');
            if (userManageBtn) userManageBtn.style.display = 'none';
        }
        if (!this.isCloud()) {
            document.querySelectorAll('[onclick*="sync"]', #syncBtn, #cloudSyncBtn, #syncStatus').forEach(el => {
                el.style.display = 'none';
            });
        }
    },
    applyLoginPermissions() {
        const edition = this._edition;
        const dropdownBtn = document.getElementById('usernameDropdownBtn');
        if (dropdownBtn) {
            dropdownBtn.style.display = this.hasUsernameDropdown() ? '' : 'none';
        }
        const rememberPwdContainer = document.getElementById('rememberPasswordContainer');
        if (rememberPwdContainer) {
            rememberPwdContainer.style.display = this.hasRememberPassword() ? 'flex' : 'none';
        }
    }
};

global.Permission = Permission;
})(typeof window !== 'undefined' ? window : this);
/* ===== 源文件: patient-archive.js ===== */
(function (global) {
    'use strict';
    function filterPrescriptionsByRange(prescriptions, range) {
        if (!Array.isArray(prescriptions)) return [];
        const now = new Date();
        let startDate = new Date(1970, 0, 1);
        switch (range) {
            case 'today':
                startDate = new Date(now.getFullYear(), now.getMonth(), now.getDate());
                break;
            case 'week':

```

```
        startDate = new Date(now.getTime() - 7 * 24 * 60 * 60 * 1000);
        break;
    case 'month':
        startDate = new Date(now.getFullYear(), now.getMonth(), 1);
        break;
    case 'quarter': {
        const quarter = Math.floor(now.getMonth() / 3);
        startDate = new Date(now.getFullYear(), quarter * 3, 1);
        break;
    }
    case 'year':
        startDate = new Date(now.getFullYear(), 0, 1);
        break;
    case 'all':
    default:
        break;
}
return prescriptions.filter(function (p) {
    const date = new Date(p.createdAt || p.date || 0);
    return date >= startDate;
});
});
}

function analyzePatients(prescriptions) {
    const list = Array.isArray(prescriptions) ? prescriptions : [];
    const patientMap = new Map();
    list.forEach(function (p) {
        const name = p.patientName || '未知';
        if (!patientMap.has(name)) {
            patientMap.set(name, {
                visits: [],
                firstVisit: null,
                lastVisit: null
            });
        }
        const patient = patientMap.get(name);
        const visitDate = new Date(p.createdAt || p.date || 0);
        patient.visits.push(visitDate);
        if (!patient.firstVisit || visitDate < patient.firstVisit) {
            patient.firstVisit = visitDate;
        }
        if (!patient.lastVisit || visitDate > patient.lastVisit) {
            patient.lastVisit = visitDate;
        }
    });
    return Array.from(patientMap.entries()).map(function (entry) {
        return {
            name: entry[0],
            visitCount: entry[1].visits.length,
            firstVisit: entry[1].firstVisit,
            lastVisit: entry[1].lastVisit
        };
    });
});
}

function analyzeDiseases(prescriptions) {
    const list = Array.isArray(prescriptions) ? prescriptions : [];
    const diseaseMap = new Map();
    list.forEach(function (p) {
        const diagnosis = p.diagnosis || '未诊断';
```

```

        diseaseMap.set(diagnosis, (diseaseMap.get(diagnosis) || 0) + 1);
    });
    return Array.from(diseaseMap.entries())
        .map(function (entry) {
            return { name: entry[0], count: entry[1] };
        })
        .sort(function (a, b) { return b.count - a.count; });
}

function analyzeMedicines(prescriptions, topN) {
    const list = Array.isArray(prescriptions) ? prescriptions : [];
    const limit = (typeof topN === 'number') ? topN : 10;
    const medicineMap = new Map();
    list.forEach(function (p) {
        if (p.items && Array.isArray(p.items)) {
            p.items.forEach(function (m) {
                const name = m.name || m.medicineName || '未知';
                const dosage = parseFloat(m.dosage) || 0;
                const doseCount = parseFloat(p.doseCount) || 1;
                medicineMap.set(name, (medicineMap.get(name) || 0) + dosage * doseCount);
            });
        }
    });
    return Array.from(medicineMap.entries())
        .map(function (entry) {
            return { name: entry[0], totalDosage: entry[1] };
        })
        .sort(function (a, b) { return b.totalDosage - a.totalDosage; })
        .slice(0, limit);
}

function analyzeVisitTrend(prescriptions) {
    const list = Array.isArray(prescriptions) ? prescriptions : [];
    const dailyMap = new Map();
    list.forEach(function (p) {
        const date = new Date(p.createdAt || p.date || 0);
        const dateStr = date.toLocaleDateString('zh-CN');
        dailyMap.set(dateStr, (dailyMap.get(dateStr) || 0) + 1);
    });
    const dates = Array.from(dailyMap.keys()).sort();
    const counts = dates.map(function (d) { return dailyMap.get(d); });
    return { dates: dates, counts: counts };
}

function analyzeMonthlyStats(prescriptions, medicines) {
    const list = Array.isArray(prescriptions) ? prescriptions : [];
    const medList = Array.isArray(medicines) ? medicines : [];
    const statsMap = new Map();
    list.forEach(function (p) {
        const date = new Date(p.createdAt || p.date || 0);
        const monthKey = date.getFullYear() + '-' + String(date.getMonth() + 1).padStart(2, '0');
        if (!statsMap.has(monthKey)) {
            statsMap.set(monthKey, {
                visits: 0,
                totalIncome: 0,
                totalRegFee: 0,
                totalCost: 0,
                totalProfit: 0,
                totalDoses: 0
            });
        }
    });
}

```

```

const stats = statsMap.get(monthKey);
stats.visits++;
stats.totalIncome += (parseFloat(p.totalAmount) || 0) + (parseFloat(p.registrationFee) || 0);
stats.totalRegFee += parseFloat(p.registrationFee) || 0;
stats.totalDoses += parseFloat(p.doseCount) || 0;
if (p.items && Array.isArray(p.items)) {
  p.items.forEach(function (item) {
    const medicine = medList.find(function (m) {
      return m.name === (item.name || item.medicineName);
    });
    const costPrice = parseFloat(medicine && medicine.costPrice) ||
      parseFloat(item.costPrice) ||
      parseFloat(item.price) || 0;
    const dosage = parseFloat(item.dosage) || 0;
    const doseCount = parseFloat(p.doseCount) || 1;
    stats.totalCost += costPrice * dosage * doseCount;
  });
}
});
statsMap.forEach(function (stats, key) {
  stats.totalProfit = stats.totalIncome - stats.totalCost;
  stats.month = key.split('-')[1];
});
return Array.from(statsMap.values())
  .sort(function (a, b) { return b.month - a.month; });
}
function formatDate(date) {
  if (!date) return '-';
  return date.toLocaleDateString('zh-CN');
}
function calculateReturnRate(patients, totalVisits) {
  const list = Array.isArray(patients) ? patients : [];
  const visits = (typeof totalVisits === 'number') ? totalVisits : 0;
  const returnPatients = list.filter(function (p) { return p.visitCount > 1; }).length;
  if (list.length === 0 || visits === 0) return 0;
  return (returnPatients / list.length) * 100;
}
function buildStatsCSV(params) {
  const p = params || {};
  const patients = Array.isArray(p.patients) ? p.patients : [];
  const diseases = Array.isArray(p.diseases) ? p.diseases : [];
  const medicines = Array.isArray(p.medicines) ? p.medicines : [];
  const totalVisits = (typeof p.totalVisits === 'number') ? p.totalVisits : 0;
  const range = p.range || 'month';
  const newPatients = patients.filter(function (x) { return x.visitCount === 1; }).length;
  const returnPatients = patients.filter(function (x) { return x.visitCount > 1; }).length;
  const returnRate = totalVisits > 0 ? ((returnPatients / patients.length) * 100) : 0;
  const rangeText = {
    today: '今日', week: '本周', month: '本月',
    quarter: '本季度', year: '本年', all: '全部'
  };
  let csv = '统计范围,' + (rangeText[range] || '全部') + '\n';
  csv += '统计时间,' + new Date().toLocaleString('zh-CN') + '\n\n';
  csv += '就诊总量,' + totalVisits + '\n';
  csv += '初诊人数,' + newPatients + '\n';
  csv += '复诊人数,' + returnPatients + '\n';
  csv += '复诊率,' + returnRate.toFixed(1) + '%\n\n';
  csv += '病种分布\n';

```

```

    csv += '病种,次数\n';
    diseases.slice(0, 10).forEach(function (d) {
        csv += ' "' + d.name + '", ' + d.count + '\n';
    });
    csv += '\n常用药材 Top 10\n';
    csv += '药材名称,总用量(g)\n';
    medicines.forEach(function (m) {
        csv += ' "' + m.name + '", ' + Math.round(m.totalDosage) + '\n';
    });
    return csv;
}

var PatientArchive = {
    filterPrescriptionsByRange: filterPrescriptionsByRange,
    analyzePatients: analyzePatients,
    analyzeDiseases: analyzeDiseases,
    analyzeMedicines: analyzeMedicines,
    analyzeVisitTrend: analyzeVisitTrend,
    analyzeMonthlyStats: analyzeMonthlyStats,
    formatDate: formatDate,
    calculateReturnRate: calculateReturnRate,
    buildStatsCSV: buildStatsCSV
};

global.PatientArchive = PatientArchive;
if (typeof module !== 'undefined' && module.exports) {
    module.exports = PatientArchive;
}

})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
/* ===== 源文件: print-utils.js ===== */
(function (global) {
    'use strict';
    const PrintUtils = {
        PAGE_SIZES: {
            A4: { width: '210mm', height: '297mm', margin: '10mm' },
            A5_PORTRAIT: { width: '148mm', height: '210mm', margin: '0' },
            A5_LANDSCAPE: { width: '210mm', height: '148mm', margin: '0' }
        },
        generatePrescriptionPrintHTML(content, orientation) {
            const isLandscape = orientation === 'landscape';
            const pageSize = isLandscape ? 'A5 landscape' : 'A5 portrait';
            const paperWidth = isLandscape ? '210mm' : '148mm';
            const paperHeight = isLandscape ? '148mm' : '210mm';
            return `<!DOCTYPE html><html><head><title>打印处方</title><meta name="viewport"
content="width=device-width, initial-scale=1.0"><style>
@page { size: ${pageSize}; margin: 0; }
body { font-family: 'SimSun', '宋体', serif; width: ${paperWidth}; padding: 0; margin: 0 auto; display: flex;
justify-content: center; align-items: flex-start; min-height: 100vh; line-height: 1.8;
-webkit-print-color-adjust: exact; print-color-adjust: exact; color: #000; }
.prescription-paper { width: ${paperWidth}; height: ${paperHeight}; padding: 28mm 15mm 20mm; margin: 0 auto;
box-sizing: border-box; }
.clinic-name { text-align: center; font-size: 16.5pt; font-weight: bold; color: #000; margin-bottom: 10px;
letter-spacing: 2px; font-family: 'KaiTi', '楷体_GB2312', '楷体', serif; }
.prescription-title { text-align: center; font-size: 16.5pt; font-weight: bold; color: #000; margin-bottom: 12px;
letter-spacing: 4px; font-family: 'KaiTi', '楷体_GB2312', '楷体', serif; }
.prescription-info { display: grid; grid-template-columns: repeat(6, 1fr); gap: 2px 2px; margin-bottom: 8px;
border-bottom: 1px solid #000; padding-bottom: 4px; font-size: 10.5pt; color: #000; }
.prescription-info > div { white-space: nowrap; text-align: left; min-width: 0; }
.prescription-info > div:nth-child(1) { grid-column: 1 / 3; }
.prescription-info > div:nth-child(2) { grid-column: 3 / 5; }

```

```

.prescription-info > div:nth-child(3) { grid-column: 5 / 7; }
.prescription-info > div:nth-child(4) { grid-column: 1 / 3; }
.prescription-info > div:nth-child(5) { grid-column: 3 / 5; }
.prescription-info > div:nth-child(6) { grid-column: 5 / 7; }
.prescription-grid { border-top: 1px solid #000; border-bottom: 1px solid #000; min-height: 200px; margin-top: 8px;
}
.prescription-grid-inner { display: grid; grid-template-columns: 1fr 1fr 1fr 1fr; gap: 2px; }
.prescription-line { padding: 5px 0; font-size: 11.25pt; text-align: center; font-weight: 500; color: #000; }
.rp-mark { font-size: 16.5pt; font-weight: bold; font-style: italic; color: #000; font-family: 'Times New Roman',
serif; }
.dose-count { text-align: right; font-size: 11.25pt; color: #000; font-weight: bold; font-family: 'Times New
Roman', serif; }
.prescription-footer { margin-top: 8px; padding-top: 4px; border-top: none; font-size: 10.5pt; color: #000; }
.usage-text { margin-bottom: 6px; font-size: 10.5pt; color: #000; font-weight: bold; white-space: nowrap; border:
none; text-decoration: none; }
.signature-row { display: flex; justify-content: space-between; font-size: 10.5pt; color: #000; }
.signature-grid { display: grid; grid-template-columns: 4fr 3fr 3fr; gap: 6px 10px; margin-top: 8px; }
.signature-item { font-size: 10.5pt; color: #000; padding: 2px 0; }
.signature-item:nth-child(1) { text-align: left; }
.signature-item:nth-child(2) { text-align: center; }
.signature-item:nth-child(3) { text-align: right; padding-right: 50%; box-sizing: border-box; }
</style></head><body><div class="prescription-paper">${content}</div></body></html>`;
    },
    printPrescription(orientation) {
        orientation = orientation || 'portrait';
        const printContent = document.getElementById('prescriptionPaper').innerHTML;
        const html = this.generatePrescriptionPrintHTML(printContent, orientation);
        if (global.AndroidNative) {
            if (global.AndroidNative.printHtml) {
                global.AndroidNative.printHtml(html);
                return;
            } else if (global.AndroidNative.invoke) {
                try {
                    global.AndroidNative.invoke('printPrescription', JSON.stringify({
                        html: html,
                        orientation: orientation
                    }));
                    return;
                } catch (e) {
                    console.error('AndroidNative打印失败:', e);
                }
            }
        }
    }
    if (global.electronAPI && global.electronAPI.printPrescription) {
        global.electronAPI.printPrescription(html, orientation);
        return;
    }
    let printFrame = document.getElementById('printFrame');
    if (!printFrame) {
        printFrame = document.createElement('iframe');
        printFrame.id = 'printFrame';
        printFrame.style.cssText = 'position:fixed;right:0;bottom:0;width:0;height:0;border:0;';
        document.body.appendChild(printFrame);
    }
    const doc = printFrame.contentWindow.document;
    doc.open();
    doc.write(html);
    doc.close();

```

```

        printFrame.contentWindow.focus();
        printFrame.contentWindow.print();
    },
    generateMedicineListHTML(medicines, clinicName) {
        const rows = medicines.map((m, i) =>
            `<tr><td>${i + 1}</td><td>${m.name || ''}</td><td>${m.code || ''}</td><td>${m.unit || ''}</td><td>${m.price || ''}</td></tr>`
        ).join('');
        return `<!DOCTYPE html><html><head><title>药材清单</title><style>
@page { size: A4; margin: 10mm; }
body { font-family: SimSun, serif; }
h2 { text-align: center; }
table { width: 100%; border-collapse: collapse; }
td, th { border: 1px solid #000; padding: 4px 8px; font-size: 12px; text-align: center; }
th { background: #f0f0f0; }
</style></head><body>
<h2>${clinicName || ''} - 药材清单</h2>
<table><thead><tr><th>#</th><th>药名</th><th>简码</th><th>单位</th><th>单价</th></tr></thead>
<tbody>${rows}</tbody></table>
</body></html>`;
    },
    exportCSV(filename, rows) {
        const csv = rows.map(row =>
            row.map(cell => {
                const val = String(cell || '');
                return val.includes(',') || val.includes('"') ? '"' + val.replace(/"/g, '""') + '"' : val;
            }).join(',')
        ).join('\n');
        const bom = '\uFEFF';
        const blob = new Blob([bom + csv], { type: 'text/csv;charset=utf-8;' });
        this.downloadBlob(blob, filename);
    },
    downloadBlob(blob, filename) {
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = filename;
        document.body.appendChild(a);
        a.click();
        document.body.removeChild(a);
        URL.revokeObjectURL(url);
    }
};

global.PrintUtils = PrintUtils;
})(typeof window !== 'undefined' ? window : this);
/* ===== 源文件: db-adapter.js ===== */
(function (global) {
    'use strict';
    let _appMode = null;
    function detectAppMode() {
        if (_appMode) return _appMode;
        if (typeof global.APP_MODE === 'string' && global.APP_MODE) {
            _appMode = global.APP_MODE;
            return _appMode;
        }
    }
    if (typeof global.CLOUD_API_BASE !== 'undefined' && global.CLOUD_API_BASE) {
        _appMode = 'cloud';
        return _appMode;
    }

```

```

    }
    if (typeof global.location !== 'undefined' && global.location.hostname) {
        if (global.location.hostname.includes('pages.dev') ||
            global.location.hostname.includes('cloud')) {
            _appMode = 'cloud';
            return _appMode;
        }
    }
    _appMode = 'offline';
    return _appMode;
}

function isCloudMode() {
    return detectAppMode() === 'cloud';
}

const DB_NAME = 'PrescriptionDB';
const DB_VERSION = 2;
const STORE_PRESCRIPTIONS = 'prescriptions';
const STORE_SETTINGS = 'settings';
const LS_KEY_PRESCRIPTIONS = 'all_prescription_list';
const LS_KEY_DELETED_IDS = 'deleted_prescription_ids';
const CLOUD_API_BASE = (typeof global.CLOUD_API_BASE !== 'undefined' && global.CLOUD_API_BASE)
    || (typeof CLOUD_API_BASE_OVERRIDE !== 'undefined' && CLOUD_API_BASE_OVERRIDE)
    || 'https://tcm-prescription-system.pages.dev/api';

let _db = null;
let _initPromise = null;

function _openDB() {
    if (_db) return Promise.resolve(_db);
    if (_initPromise) return _initPromise;
    _initPromise = new Promise((resolve, reject) => {
        try {
            if (typeof indexedDB === 'undefined') {
                reject(new Error('IndexedDB not available'));
                return;
            }
            const request = indexedDB.open(DB_NAME, DB_VERSION);
            request.onupgradeneeded = function (event) {
                const database = event.target.result;
                if (!database.objectStoreNames.contains(STORE_PRESCRIPTIONS)) {
                    database.createObjectStore(STORE_PRESCRIPTIONS, { keyPath: 'id' });
                }
                if (!database.objectStoreNames.contains(STORE_SETTINGS)) {
                    database.createObjectStore(STORE_SETTINGS, { keyPath: 'key' });
                }
            };
            request.onsuccess = function (event) {
                _db = event.target.result;
                resolve(_db);
            };
            request.onerror = function (event) {
                console.error('[DbAdapter] IndexedDB 打开失败:', event.target.error);
                reject(event.target.error);
            };
        } catch (e) {
            reject(e);
        }
    });
    return _initPromise;
}

```

```
function _tx(storeName, mode) {
  return _openDB().then(db => {
    const transaction = db.transaction([storeName], mode);
    return transaction.objectStore(storeName);
  });
}

function _wrapRequest(request) {
  return new Promise((resolve, reject) => {
    request.onsuccess = () => resolve(request.result);
    request.onerror = () => reject(request.error);
  });
}

async function _idbPutPrescription(record) {
  const store = await _tx(STORE_PRESCRIPTIONS, 'readwrite');
  await _wrapRequest(store.put(record));
}

async function _idbPutAllPrescriptions(records) {
  const db = await _openDB();
  return new Promise((resolve, reject) => {
    const transaction = db.transaction([STORE_PRESCRIPTIONS], 'readwrite');
    const store = transaction.objectStore(STORE_PRESCRIPTIONS);
    for (const r of records) store.put(r);
    transaction.oncomplete = () => resolve();
    transaction.onerror = () => reject(transaction.error);
  });
}

async function _idbDeletePrescription(id) {
  const store = await _tx(STORE_PRESCRIPTIONS, 'readwrite');
  await _wrapRequest(store.delete(id));
}

async function _idbGetAllPrescriptions() {
  try {
    const store = await _tx(STORE_PRESCRIPTIONS, 'readonly');
    const result = await _wrapRequest(store.getAll());
    return result || [];
  } catch (e) {
    console.warn('[DbAdapter] IndexedDB 读取失败:', e);
    return [];
  }
}

async function _idbClearPrescriptions() {
  const store = await _tx(STORE_PRESCRIPTIONS, 'readwrite');
  await _wrapRequest(store.clear());
}

async function _idbGetSetting(key) {
  try {
    const store = await _tx(STORE_SETTINGS, 'readonly');
    const result = await _wrapRequest(store.get(key));
    return result ? result.value : null;
  } catch (e) {
    return null;
  }
}

async function _idbPutSetting(key, value) {
  const store = await _tx(STORE_SETTINGS, 'readwrite');
  await _wrapRequest(store.put({ key, value }));
}

function _lsGet(key) {
```

```
try { return global.localStorage ? global.localStorage.getItem(key) : null; }
catch (e) { return null; }
}
function _lsSet(key, value) {
  try { if (global.localStorage) global.localStorage.setItem(key, value); }
  catch (e) { console.warn('[DbAdapter] localStorage 写入失败:', e); }
}
function _lsRemove(key) {
  try { if (global.localStorage) global.localStorage.removeItem(key); }
  catch (e) { }
}
function _lsGetPrescriptions() {
  const raw = _lsGet(LS_KEY_PRESCRIPTIONS);
  if (!raw) return [];
  try {
    const data = JSON.parse(raw);
    return Array.isArray(data) ? data : [];
  } catch (e) { return []; }
}
function _lsSetPrescriptions(arr) {
  _lsSet(LS_KEY_PRESCRIPTIONS, JSON.stringify(arr));
}
function _lsGetDeletedIds() {
  const raw = _lsGet(LS_KEY_DELETED_IDS);
  if (!raw) return [];
  try {
    const data = JSON.parse(raw);
    return Array.isArray(data) ? data : [];
  } catch (e) { return []; }
}
function _lsAddDeletedId(id) {
  const ids = _lsGetDeletedIds();
  const idStr = String(id);
  if (!ids.includes(idStr)) {
    ids.push(idStr);
    if (ids.length > 500) ids.shift();
    _lsSet(LS_KEY_DELETED_IDS, JSON.stringify(ids));
  }
}
function _hasElectron() {
  return !(global.electronAPI && global.electronAPI.isElectron && global.electronAPI.getUserData);
}
async function _electronGet(key) {
  if (!_hasElectron()) return null;
  try {
    const result = await global.electronAPI.getUserData(key);
    if (result && result.success && result.data !== null) {
      return typeof result.data === 'string' ? result.data : JSON.stringify(result.data);
    }
  } catch (e) { }
  return null;
}
async function _electronSet(key, value) {
  if (!_hasElectron()) return false;
  try {
    if (global.electronAPI.saveUserData) {
      await global.electronAPI.saveUserData(key, value);
      return true;
    }
  }
}
```

```

    }
  } catch (e) { }
  return false;
}
function _getAuthToken() {
  try {
    const session = global.sessionStorage && global.sessionStorage.getItem('auth:currentUser');
    if (session) {
      const user = JSON.parse(session);
      return user && user.token ? user.token : null;
    }
    const local = global.localStorage && global.localStorage.getItem('auth:currentUser');
    if (local) {
      const user = JSON.parse(local);
      return user && user.token ? user.token : null;
    }
  } catch (e) { }
  return null;
}
async function _cloudFetch(path, options) {
  if (typeof global.cloudFetch === 'function') {
    const url = path.startsWith('http') ? path : CLOUD_API_BASE + path;
    return await global.cloudFetch(url, options);
  }
  options = options || {};
  options.headers = options.headers || {};
  options.headers['Content-Type'] = 'application/json';
  const token = _getAuthToken();
  if (token) options.headers['Authorization'] = 'Bearer ' + token;
  try {
    const response = await fetch(CLOUD_API_BASE + path, options);
    if (!response.ok) {
      console.warn('[DbAdapter] 云端 API HTTP ' + response.status);
    }
    return await response.json();
  } catch (e) {
    console.warn('[DbAdapter] 云端 API 请求失败:', e);
    return { success: false, error: e.message, offline: true };
  }
}
const DbAdapter = {
  async init() {
    try { await _openDB(); } catch (e) { }
    return this;
  },
  async savePrescription(record, options) {
    options = options || {};
    if (options.cloud === undefined) options.cloud = isCloudMode();
    const now = new Date().toISOString();
    const normalized = {
      ...record,
      id: record.id || Date.now(),
      createdAt: record.createdAt || now,
      updatedAt: now,
      createdBy: record.createdBy || options.username || 'unknown',
      userRole: record.userRole || options.userRole,
      isAdmin: record.isAdmin !== undefined ? record.isAdmin : options.isAdmin
    };
  }
};

```

```

    if (options.cloud) {
      const result = await _cloudFetch('/prescriptions', {
        method: 'POST',
        body: JSON.stringify({ prescription: normalized })
      });
      if (result && result.success) {
        if (result.savedPrescription) {
          try { await _idbPutPrescription(result.savedPrescription); } catch (e) { }
        }
        return { success: true, savedPrescription: result.savedPrescription, data: result.data };
      }
      console.warn('[DbAdapter] 云端保存失败，降级到本地:', result && result.error);
    }
    try {
      await _idbPutPrescription(normalized);
    } catch (e) {
      console.warn('[DbAdapter] IndexedDB 保存失败，降级到 localStorage:', e);
      const all = _lsGetPrescriptions();
      const idx = all.findIndex(p => String(p.id) === String(normalized.id));
      if (idx >= 0) all[idx] = normalized;
      else all.unshift(normalized);
      _lsSetPrescriptions(all);
    }
    if (_hasElectron()) {
      try {
        const all = await this.getAllPrescriptions();
        await _electronSet(LS_KEY_PRESCRIPTIONS, JSON.stringify(all));
      } catch (e) { }
    }
    return { success: true, savedPrescription: normalized, offline: !options.cloud };
  },
  async getAllPrescriptions(options) {
    options = options || {};
    if (options.cloud === undefined) options.cloud = isCloudMode();
    let result = [];
    if (options.cloud) {
      const resp = await _cloudFetch('/prescriptions', { method: 'GET' });
      if (resp && resp.success && Array.isArray(resp.data)) {
        result = resp.data;
      }
    }
    if (result.length === 0) {
      result = await _idbGetAllPrescriptions();
    }
    if (result.length === 0) {
      result = _lsGetPrescriptions();
      if (result.length > 0) {
        try { await _idbPutAllPrescriptions(result); } catch (e) { }
      }
    }
    if (result.length === 0) {
      const dbData = await _idbGetSetting(LS_KEY_PRESCRIPTIONS);
      if (dbData) {
        const data = typeof dbData === 'string' ? JSON.parse(dbData) : dbData;
        if (Array.isArray(data)) {
          result = data;
          _lsSetPrescriptions(result);
          try { await _idbPutAllPrescriptions(result); } catch (e) { }
        }
      }
    }
  }
}

```

```

    }
  }
}
if (result.length === 0 && _hasElectron()) {
  const rawData = await _electronGet(LS_KEY_PRESCRIPTIONS);
  if (rawData) {
    try {
      const data = typeof rawData === 'string' ? JSON.parse(rawData) : rawData;
      if (Array.isArray(data)) {
        result = data;
        _lsSetPrescriptions(result);
        try { await _idbPutAllPrescriptions(result); } catch (e) { }
      }
    } catch (e) { }
  }
}
if (options.filterUsername) {
  result = result.filter(p => p.createdBy === options.filterUsername);
}
if (!options.includeDeleted) {
  const deletedIds = _lsGetDeletedIds();
  if (deletedIds.length > 0) {
    result = result.filter(p => !deletedIds.includes(String(p.id)));
  }
}
result.sort((a, b) => {
  const timeA = a.createdAt ? new Date(a.createdAt).getTime() : (a.id || 0);
  const timeB = b.createdAt ? new Date(b.createdAt).getTime() : (b.id || 0);
  return timeB - timeA;
});
return result;
},
async getPrescriptionById(id) {
  try {
    const store = await _tx(STORE_PRESCRIPTIONS, 'readonly');
    const result = await _wrapRequest(store.get(id));
    if (result) return result;
  } catch (e) { }
  const all = _lsGetPrescriptions();
  return all.find(p => String(p.id) === String(id)) || null;
},
async deletePrescription(id, options) {
  options = options || {};
  if (options.cloud === undefined) options.cloud = isCloudMode();
  const idStr = String(id);
  if (options.cloud) {
    const permanentParam = options.permanent ? '&permanent=true' : '';
    const result = await _cloudFetch('/prescriptions?id=' + idStr + permanentParam, {
      method: 'DELETE'
    });
    if (result && result.success) {
      try { await _idbDeletePrescription(id); } catch (e) { }
      return { success: true, softDeleted: !options.permanent };
    }
    console.warn('[DbAdapter] 云端删除失败，降级到本地:', result && result.error);
  }
  try { await _idbDeletePrescription(id); } catch (e) { }
  const all = _lsGetPrescriptions();

```

```

    const filtered = all.filter(p => String(p.id) !== idStr);
    _lsSetPrescriptions(filtered);
    if (!options.permanent) {
        _lsAddDeletedId(id);
    }
    return { success: true, offline: true };
},
async saveAllPrescriptions(records, options) {
    options = options || {};
    if (options.cloud === undefined) options.cloud = isCloudMode();
    if (!Array.isArray(records)) records = [records];
    if (options.cloud) {
        const result = await _cloudFetch('/prescriptions', {
            method: 'POST',
            body: JSON.stringify({ prescription: records })
        });
        if (result && result.success) {
            return { success: true, data: result.data };
        }
    }
    try { await _idbPutAllPrescriptions(records); } catch (e) { }
    _lsSetPrescriptions(records);
    return { success: true, offline: !options.cloud };
},
async clearAllPrescriptions() {
    try { await _idbClearPrescriptions(); } catch (e) { }
    _lsRemove(LS_KEY_PRESCRIPTIONS);
    return { success: true };
},
async getSetting(key) {
    let value = await _idbGetSetting(key);
    if (value !== null && value !== undefined) return value;
    return _lsGet(key);
},
async setSetting(key, value) {
    try { await _idbPutSetting(key, value); } catch (e) { }
    const strValue = typeof value === 'string' ? value : JSON.stringify(value);
    _lsSet(key, strValue);
    if (_hasElectron()) await _electronSet(key, strValue);
},
getDeletedIds() { return _lsGetDeletedIds(); },
addDeletedId(id) { _lsAddDeletedId(id); },
clearDeletedIds() { _lsRemove(LS_KEY_DELETED_IDS); },
async migrateFromLegacyStorage() {
    const existing = await _idbGetAllPrescriptions();
    if (existing.length > 0) return 0;
    let migrated = _lsGetPrescriptions();
    if (migrated.length === 0) {
        const dbData = await _idbGetSetting(LS_KEY_PRESCRIPTIONS);
        if (dbData) {
            const data = typeof dbData === 'string' ? JSON.parse(dbData) : dbData;
            if (Array.isArray(data)) migrated = data;
        }
    }
    if (migrated.length === 0 && _hasElectron()) {
        const raw = await _electronGet(LS_KEY_PRESCRIPTIONS);
        if (raw) {
            try {

```

```

        const data = typeof raw === 'string' ? JSON.parse(raw) : raw;
        if (Array.isArray(data)) migrated = data;
    } catch (e) { }
    }
}
if (migrated.length > 0) {
    try { await _idbPutAllPrescriptions(migrated); } catch (e) { }
    _lsSetPrescriptions(migrated);
    return migrated.length;
}
return 0;
},
async diagnose() {
    const status = {
        appMode: detectAppMode(), // ★v2.0 新增
        indexedDB: false,
        localStorage: false,
        electron: false,
        cloud: false,
        cloudModule: typeof global.cloudFetch === 'function', // ★v2.0 新增
        prescriptionCount: 0
    };
    try {
        await _openDB();
        status.indexedDB = true;
    } catch (e) { }
    status.localStorage = !!global.localStorage;
    status.electron = _hasElectron();
    status.prescriptionCount = (await this.getAllPrescriptions()).length;
    return status;
}
};
global.DbAdapter = DbAdapter;
global.DbAdapter.getAppMode = detectAppMode;
global.DbAdapter.isCloudMode = isCloudMode;
if (typeof window !== 'undefined') {
    DbAdapter.init().catch(e => console.warn('[DbAdapter] 自动初始化失败:', e));
}
})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
/* ===== 源文件: performance-utils.js ===== */
(function (global) {
    'use strict';
    function debounce(fn, wait) {
        let timer = null;
        const debounced = function () {
            const ctx = this, args = arguments;
            if (timer) clearTimeout(timer);
            timer = setTimeout(() => {
                fn.apply(ctx, args);
                timer = null;
            }, wait || 200);
        };
        debounced.cancel = function () {
            if (timer) { clearTimeout(timer); timer = null; }
        };
        debounced.flush = function () {
            if (timer) {
                clearTimeout(timer);
            }
        };
    }

```

```
        fn.apply(this, arguments);
        timer = null;
    }
};
return debounced;
}
function throttle(fn, wait) {
    let lastTime = 0;
    let timer = null;
    return function () {
        const ctx = this, args = arguments;
        const now = Date.now();
        const remaining = wait - (now - lastTime);
        if (remaining <= 0) {
            if (timer) { clearTimeout(timer); timer = null; }
            lastTime = now;
            fn.apply(ctx, args);
        } else if (!timer) {
            timer = setTimeout(() => {
                lastTime = Date.now();
                timer = null;
                fn.apply(ctx, args);
            }, remaining);
        }
    };
}
}
function runIdle(fn, options) {
    if (typeof global.requestIdleCallback === 'function') {
        return global.requestIdleCallback(fn, options || { timeout: 1000 });
    }
    return setTimeout(fn, 50);
}
function cancelIdle(id) {
    if (typeof global.cancelIdleCallback === 'function') {
        global.cancelIdleCallback(id);
    } else {
        clearTimeout(id);
    }
}
}
function createVirtualScroller(options) {
    const config = Object.assign({
        container: null,
        itemHeight: 50,
        bufferSize: 5,
        render: () => document.createElement('div'),
        onScroll: null
    }, options);
    if (!config.container) {
        console.warn('[PerfUtils] createVirtualScroller: container is required');
        return null;
    }
    const state = {
        data: [],
        scrollTop: 0,
        visibleCount: 0,
        renderTimer: null
    };
    const viewport = config.container;
```

```

viewport.style.overflowY = 'auto';
viewport.style.position = 'relative';
const spacer = document.createElement('div');
spacer.style.position = 'relative';
viewport.appendChild(spacer);
const content = document.createElement('div');
content.style.position = 'absolute';
content.style.top = '0';
content.style.left = '0';
content.style.right = '0';
spacer.appendChild(content);
function render() {
  const containerHeight = viewport.clientHeight;
  state.visibleCount = Math.ceil(containerHeight / config.itemHeight) + config.bufferSize * 2;
  const startIndex = Math.max(0, Math.floor(state.scrollTop / config.itemHeight) - config.bufferSize);
  const endIndex = Math.min(state.data.length, startIndex + state.visibleCount);
  spacer.style.height = (state.data.length * config.itemHeight) + 'px';
  content.style.transform = 'translateY(' + (startIndex * config.itemHeight) + 'px)';
  content.innerHTML = '';
  for (let i = startIndex; i < endIndex; i++) {
    const el = config.render(state.data[i], i);
    if (el) {
      el.style.height = config.itemHeight + 'px';
      el.style.boxSizing = 'border-box';
      content.appendChild(el);
    }
  }
  if (config.onScroll) config.onScroll(startIndex, endIndex);
}
function onScroll() {
  state.scrollTop = viewport.scrollTop;
  if (state.renderTimer) cancelAnimationFrame(state.renderTimer);
  state.renderTimer = requestAnimationFrame(render);
}
viewport.addEventListener('scroll', throttle(onScroll, 16), { passive: true });
return {
  setData(items) {
    state.data = items || [];
    state.scrollTop = viewport.scrollTop = 0;
    render();
  },
  refresh() { render(); },
  scrollToIndex(index) {
    viewport.scrollTop = index * config.itemHeight;
  },
  destroy() {
    viewport.removeEventListener('scroll', onScroll);
    viewport.innerHTML = '';
  }
};
}
let _lazyObserver = null;
function lazyImage(img) {
  if (!img) return;
  if (_lazyObserver) {
    _lazyObserver.observe(img);
  } else {
    const src = img.getAttribute('data-src');

```

```

        if (src) img.src = src;
    }
}
function observeLazyImages(images) {
    if (!images || !images.length) return;
    if (!('IntersectionObserver' in global)) {
        images.forEach(img => {
            const src = img.getAttribute('data-src');
            if (src) img.src = src;
        });
        return;
    }
    if (!_lazyObserver) {
        _lazyObserver = new IntersectionObserver((entries) => {
            entries.forEach(entry => {
                if (entry.isIntersecting) {
                    const img = entry.target;
                    const src = img.getAttribute('data-src');
                    if (src) {
                        img.src = src;
                        img.removeAttribute('data-src');
                    }
                    _lazyObserver.unobserve(img);
                }
            });
        }, {
            rootMargin: '50px 0px', // 提前 50px 加载
            threshold: 0.01
        });
    }
    images.forEach(img => {
        if (img.getAttribute('data-src')) {
            _lazyObserver.observe(img);
        }
    });
}
function batchAppend(container, items, createElement) {
    if (!container || !items || !items.length) return;
    const fragment = document.createDocumentFragment();
    items.forEach(item => {
        const el = createElement(item);
        if (el) fragment.appendChild(el);
    });
    container.appendChild(fragment);
}
function prefetch(url, as) {
    runIdle(() => {
        try {
            const link = document.createElement('link');
            link.rel = 'prefetch';
            link.href = url;
            link.as = as || 'fetch';
            document.head.appendChild(link);
        } catch (e) { }
    });
}
function measure(name, fn) {
    const start = performance.now();

```

```
const result = fn();
const duration = performance.now() - start;
if (duration > 16) { // 超过一帧才记录
  console.log('[Perf] ' + name + ': ' + duration.toFixed(2) + 'ms');
}
return result;
}
}
async function measureAsync(name, fn) {
  const start = performance.now();
  try {
    return await fn();
  } finally {
    const duration = performance.now() - start;
    if (duration > 16) {
      console.log('[Perf] ' + name + ': ' + duration.toFixed(2) + 'ms');
    }
  }
}
}
const PerfUtils = {
  debounce,
  throttle,
  runIdle,
  cancelIdle,
  createVirtualScroller,
  lazyImage,
  observeLazyImages,
  batchAppend,
  prefetch,
  measure,
  measureAsync
};
global.PerfUtils = PerfUtils;
function setupDropdownKeyboardFix() {
  return;
  if (window.innerWidth >= 769) return;
  var dropdown = document.getElementById('medicineSearchDropdown');
  if (!dropdown) {
    setTimeout(setupDropdownKeyboardFix, 500);
    return;
  }
  function repositionDropdown() {
    if (dropdown.style.display !== 'block') return;
    var activeEl = document.activeElement;
    if (!activeEl || activeEl.tagName !== 'INPUT') return;
    var rect = activeEl.getBoundingClientRect();
    var winHeight = window.innerHeight;
    var vvHeight = window.visualViewport ? window.visualViewport.height : winHeight;
    var visibleHeight;
    if (winHeight - vvHeight < 50) {
      visibleHeight = winHeight * 0.6;
    } else {
      visibleHeight = vvHeight;
    }
    var availableBelow = visibleHeight - rect.bottom;
    var availableAbove = rect.top;
    var dropdownMaxHeight = 250;
    if (availableBelow < 120 && availableAbove > availableBelow) {
      var aboveHeight = Math.min(availableAbove - 4, dropdownMaxHeight);
    }
  }
}
```

```

        dropdown.style.top = Math.max(0, rect.top - aboveHeight - 2) + 'px';
        dropdown.style.maxHeight = aboveHeight + 'px';
    } else {
        var belowHeight = Math.min(Math.max(availableBelow - 4, 60), dropdownMaxHeight);
        dropdown.style.top = (rect.bottom + 2) + 'px';
        dropdown.style.maxHeight = belowHeight + 'px';
    }
}

var observer = new MutationObserver(function(mutations) {
    mutations.forEach(function(m) {
        if (m.attributeName === 'style' && dropdown.style.display === 'block') {
            setTimeout(repositionDropdown, 10);
            setTimeout(repositionDropdown, 100);
            setTimeout(repositionDropdown, 300);
        }
    });
});

observer.observe(dropdown, { attributes: true, attributeFilter: ['style'] });
window.addEventListener('resize', function() {
    if (dropdown.style.display === 'block') {
        setTimeout(repositionDropdown, 50);
        setTimeout(repositionDropdown, 200);
    }
});

if (window.visualViewport) {
    window.visualViewport.addEventListener('resize', function() {
        if (dropdown.style.display === 'block') {
            setTimeout(repositionDropdown, 30);
            setTimeout(repositionDropdown, 150);
        }
    });
}

}

function setupInputAutocompleteOff() {
    if (window.innerWidth >= 769) return;
    function setOff(el) {
        if (!el || el.tagName !== 'INPUT' && el.tagName !== 'TEXTAREA') return;
        if (el.__bnAutocompleteOff) return;
        el.__bnAutocompleteOff = true;
        el.setAttribute('autocomplete', 'off');
        el.setAttribute('autocorrect', 'off');
        el.setAttribute('autocapitalize', 'off');
        el.setAttribute('spellcheck', 'false');
        var type = el.getAttribute('type') || '';
        var id = el.getAttribute('id') || '';
        var className = el.className || '';
        if (type === 'number' || id.indexOf('Age') >= 0 || id.indexOf('age') >= 0 ||
            id.indexOf('Fee') >= 0 || id.indexOf('fee') >= 0 || id.indexOf('clinicNo') >= 0 ||
            id.indexOf('dose') >= 0 || id.indexOf('Dose') >= 0 || id.indexOf('Count') >= 0) {
            el.setAttribute('inputmode', 'numeric');
        } else if (type === 'tel' || id.indexOf('Phone') >= 0 || id.indexOf('phone') >= 0) {
            el.setAttribute('inputmode', 'tel');
        } else if (type === 'text') {
            el.setAttribute('inputmode', 'search');
        }
    }
}

var inputs = document.querySelectorAll('input,textarea');
for (var i = 0; i < inputs.length; i++) {

```

```

        setOff(inputs[i]);
    }
    var observer = new MutationObserver(function(mutations) {
        mutations.forEach(function(m) {
            if (m.addedNodes) {
                for (var i = 0; i < m.addedNodes.length; i++) {
                    var node = m.addedNodes[i];
                    if (node.nodeType === 1) {
                        setOff(node);
                        var children = node.querySelectorAll('input,textarea');
                        for (var j = 0; j < children.length; j++) {
                            setOff(children[j]);
                        }
                    }
                }
            }
        });
    });
    observer.observe(document.body || document.documentElement, { childList: true, subtree: true });
}
function setupMobileKeyboardScroll() {
    if (window.innerWidth >= 769) return;
    var lastFocusedInput = null;
    function doScroll(target) {
        if (!target) return;
        try {
            var vvH = window.visualViewport ? window.visualViewport.height : window.innerHeight;
            var safeMargin = 60;
            target.scrollIntoView({ block: 'center', behavior: 'smooth' });
            var container = target.closest ? target.closest('.medicine-table-container') : null;
            if (container) {
                var targetRect = target.getBoundingClientRect();
                var containerRect = container.getBoundingClientRect();
                var vh = vvH; // adjustNothing模式: 使用visualViewport高度
                if (targetRect.bottom > vh - safeMargin) {
                    var targetCenterInContainer = targetRect.top + targetRect.height / 2 - containerRect.top;
                    var containerCenter = containerRect.height / 2;
                    var scrollDelta = targetCenterInContainer - containerCenter;
                    if (Math.abs(scrollDelta) > 5) {
                        var newTop = container.scrollTop + scrollDelta;
                        var maxScroll = container.scrollHeight - container.clientHeight;
                        if (newTop < 0) newTop = 0;
                        if (newTop > maxScroll) newTop = maxScroll;
                        try {
                            container.scrollTo({ top: newTop, behavior: 'smooth' });
                        } catch(e) {
                            container.scrollTop = newTop;
                        }
                    }
                }
            }
        } catch(err) {
            try { target.scrollIntoView(false); } catch(e2) {}
        }
    }
    document.addEventListener('focusin', function(e) {
        var target = e.target;
        if (!target) return;
    });
}

```

```

    if (target.tagName !== 'INPUT' && target.tagName !== 'TEXTAREA' && target.tagName !== 'SELECT') return;
    var table = target.closest ? target.closest('.medicine-table') : null;
    if (!table) return;
    lastFocusedInput = target;
    setTimeout(function() { doScroll(target); }, 100);
    setTimeout(function() { doScroll(target); }, 300);
    setTimeout(function() { doScroll(target); }, 500);
    setTimeout(function() { doScroll(target); }, 800);
  }, true);
document.addEventListener('focusout', function() {
  setTimeout(function() {
    if (!document.activeElement || document.activeElement === document.body) {
      lastFocusedInput = null;
    }
  }, 100);
}, true);
function onViewportChange() {
  if (lastFocusedInput) {
    setTimeout(function() { doScroll(lastFocusedInput); }, 50);
    setTimeout(function() { doScroll(lastFocusedInput); }, 200);
    setTimeout(function() { doScroll(lastFocusedInput); }, 400);
  }
}
if (window.visualViewport) {
  window.visualViewport.addEventListener('resize', onViewportChange);
  window.visualViewport.addEventListener('scroll', onViewportChange);
}
}
if (document.readyState === 'loading') {
  document.addEventListener('DOMContentLoaded', setupMobileKeyboardScroll);
  document.addEventListener('DOMContentLoaded', setupDropdownKeyboardFix);
  document.addEventListener('DOMContentLoaded', setupInputAutocompleteOff);
} else {
  setupMobileKeyboardScroll();
  setupDropdownKeyboardFix();
  setupInputAutocompleteOff();
}
})(typeof window !== 'undefined' ? window : (typeof globalThis !== 'undefined' ? globalThis : this));
/* ===== 源文件: debug-logger.js ===== */
(function (global) {
  'use strict';
  const SENSITIVE_KEY_RE = /(password|pwd|passwd|token|secret|activation|authcode|auth_code|apikey|api_key|accesskey|access_key|cookie|credential|authorization|signature|privatekey|private_key)/i;
  const DBG = {
    _enabled: false,
    _version: 'unknown',
    _device: 'unknown',
    init() {
      this._enabled = (
        location.search.includes('debug') ||
        localStorage.getItem('debug') === '1' ||
        (global.electronAPI && global.electronAPI.isDebugMode)
      );
      const ua = navigator.userAgent || '';
      if (/Android/i.test(ua)) {
        this._device = 'Android';
      } else if (/Win/i.test(ua)) {
        this._device = 'Windows';
      }
    }
  };

```

```

    } else if (/Mac/i.test(ua)) {
        this._device = 'macOS';
    } else if (/Linux/i.test(ua)) {
        this._device = 'Linux';
    } else {
        this._device = 'Unknown';
    }
    if (global.Permission && Permission._edition) {
        this._version = Permission._edition;
    } else if (typeof CONFIG !== 'undefined' && CONFIG && CONFIG.edition) {
        this._version = CONFIG.edition;
    } else if (global.EDITION) {
        this._version = global.EDITION;
    }
},
_redact(value, depth) {
    depth = depth || 0;
    if (depth > 6) return '[depth]';
    if (value === null || value === undefined) return value;
    if (typeof value === 'string') return value;
    if (Array.isArray(value)) return value.map(v => this._redact(v, depth + 1));
    if (typeof value === 'object') {
        const out = {};
        for (const k of Object.keys(value)) {
            if (SENSITIVE_KEY_RE.test(k)) {
                const v = value[k];
                out[k] = (v === null || v === undefined) ? v : '***';
            } else {
                out[k] = this._redact(value[k], depth + 1);
            }
        }
        return out;
    }
    return value;
},
_now() {
    const d = new Date();
    const pad = n => String(n).padStart(2, '0');
    return pad(d.getHours()) + ':' + pad(d.getMinutes()) + ':' + pad(d.getSeconds());
},
_format(module, action) {
    return `[DBG][${this._now()}][${this._version}][${this._device}][${module}] ${action}`;
},
log(module, action, data) {
    if (!this._enabled) return;
    this.init();
    console.log(this._format(module, action), this._redact(data) || '');
},
warn(module, action, data) {
    this.init();
    console.warn(this._format(module, action), this._redact(data) || '');
},
error(module, action, data) {
    this.init();
    console.error(this._format(module, action), this._redact(data) || '');
},
wrap(module, fn, action) {
    const self = this;

```

```

return function (...args) {
  self.log(module, action || fn.name || 'anonymous', { args: args.length });
  try {
    const result = fn.apply(this, args);
    if (result && typeof result.then === 'function') {
      return result.then(r => {
        self.log(module, (action || fn.name) + '.done', { success: true });
        return r;
      }).catch(e => {
        self.error(module, (action || fn.name) + '.error', e.message);
        throw e;
      });
    }
    self.log(module, (action || fn.name) + '.done', { success: true });
    return result;
  } catch (e) {
    self.error(module, (action || fn.name) + '.error', e.message);
    throw e;
  }
};

}

};

DBG.init();
global.DBG = DBG;
})(typeof window !== 'undefined' ? window : this);
/* ===== 源文件: afterPack.js ===== */
const asarmor = require('asarmor');
const { join } = require('path');
const fs = require('fs');
const BLOAT_GB = parseInt(process.env.ASARMOR_BLOAT_GB || '100', 10);
exports.default = async ({ appOutDir, packager }) => {
  const resourcesDir = packager.getResourcesDir(appOutDir);
  try {
    const asarPath = join(resourcesDir, 'app.asar');
    console.log('[asarmor] Applying patches to ' + asarPath);
    console.log('[asarmor] Bloat size: ' + BLOAT_GB + ' GB (only affects extraction, not archive size)');
    const archive = await asarmor.open(asarPath);
    const bloatPatch = asarmor.createBloatPatch(BLOAT_GB);
    archive.patch(bloatPatch);
    await archive.write(asarPath);
    console.log('[asarmor] ASAR protection applied successfully (patch + bloat)');
  } catch (err) {
    console.error('[asarmor] Error applying patches:', err.message);
  }
  try {
    const peGuard = require('.././../shared/pe-guard.cjs');
    const productFilename = packager.appInfo.productFilename;
    const mainExe = join(appOutDir, productFilename + '.exe');
    if (fs.existsSync(mainExe)) {
      const r = peGuard.embedZone(mainExe);
      console.log('[pe-guard] .bnzc 完整性区段嵌入成功 (mode=' + r.mode + ') -> ' + mainExe);
    } else {
      console.warn('[pe-guard] 未找到主 exe: ' + mainExe + ', 跳过 .bnzc 嵌入');
    }
  } catch (err) {
    console.warn('[pe-guard] PE 完整性区段嵌入失败 (非阻塞): ' + err.message);
  }
};

```